Rawad
Digital Egypt Pioneers

ITC
Information Technology Institute

العالمية
AL-ALAMIA

SYSTEM DESIGN DOCUMENT · v1.1

System Design Document

A managed platform that runs digital-service projects between business owners and freelancers, from the first request to delivery and the financial ledger.

Date	Delivery	Backend	Frontend
28 October 2026 Project kick-off	Saturday 5 December 2026	ASP.NET Core Web API .NET 10 · SQL Server	React · TypeScript Arabic / English (RTL / LTR)

Team

Mohamed Tawfik
Full Stack

Menna Reda
Full Stack

Abdelrahman Khaled
Full Stack

Merna Abdelfattah
Full Stack

Nahed Magdy
Full Stack

Contents

1	Executive summary
2	Scope: what we build and what we defer
3	Actors and the project journey
4	Business rules
5	System architecture
6	State machines
7	Critical flows: first accept, monthly cap, deductions and blocks
8	Database design and ERD changes
9	API design
10	UI design and brand
11	Performance, security and quality
12	Delivery plan and team ownership
13	Risks and definition of done
14	Future work
A	Appendix: glossary and enum values

How to read this document. Sections 1 to 4 explain the idea and its rules for any reader. Sections 5 to 9 are the technical reference the team will use day to day. Section 10 shows the visual prototype in the final brand. Sections 12 and 13 lay out the plan week by week. Anything beyond the agreed scope lives in section 14 and is not built now.

1 Executive summary

Al-Alamia is a managed outsourcing platform. A business owner never negotiates with a freelancer and never browses an open marketplace. They ask Al-Alamia for a digital service (a website, a store, a system, marketing work), and Al-Alamia reviews the request, prices it, breaks it into tasks, hands those tasks to approved freelancers and checks every delivery before it reaches the client.

The problem

The owner of a restaurant, a café or a small company needs digital work done, but cannot judge a freelancer's quality, manage a deadline or review a delivery. A talented freelancer, on the other side, loses time hunting for work, negotiating and chasing payment. Open marketplaces push all of that onto both parties.

The solution

A middleman with explicit rules: **pre-approved freelancers**, **tasks with a clear track, skills and deadline**, **the first valid accept wins**, **admin review of every delivery**, and a **financial ledger** that records earnings and deductions. The client follows the project from one screen; the freelancer focuses on the work.

3	40	10	5.5
Roles: client, freelancer, admin (through Identity roles)	Entities in one database, grouped into 12 modules	Accepted tasks per freelancer per month, at most	Weeks to deliver: 28 Oct to 5 Dec 2026

What sets the design apart

- **Conflict-free assignment.** One task is never given to two freelancers, however busy the platform gets. The guarantee sits in the database, not only in application code (section 7).
- **Tunable policy.** The monthly cap, the block durations and the deduction percentage are configuration values, not numbers buried in code.
- **A full audit trail.** Every status change on a project or task is stored with who made it, when, and why.
- **A disciplined scope.** The core is built carefully; every extra idea goes to the future-work list.

2 Scope: what we build and what we defer

The governing rule is simple: **no new features once the plan starts**. The time goes into code quality, performance and tests.

In scope MVP

- Sign-up and login with three roles; JWT with refresh tokens
- Freelancer approval, track and skills, portfolio (exactly 3 images)
- Service catalog, project creation, customizations, meeting requests
- Admin pricing, task creation and publishing, task offers
- First-accept assignment with concurrency protection; monthly cap
- Delivery and review (accept / revise / final reject); revisions
- 20% late deduction, blocks and reassignment
- Financial ledger (ledger only)
- "Track your presence" marketing package with a snapshot copy
- Support tickets; in-app and e-mail notifications
- Arabic and English UI, RTL and LTR

Out of scope Future work

- Real payments and payouts (ledger only for now)
- Skill assessment tests before approval
- Real-time notifications (SignalR) and live chat
- Mutual ratings and freelancer ranking
- Native mobile app
- Smart task-matching assistant
- Advanced analytics and exportable reports
- Multi-currency and tax invoicing

Details and priorities are in section 14.

What does "ledger only" mean? The platform records earnings and deductions and shows them, but it never moves real money. The "Payouts" screen in the prototype should be read that way: a view of the ledger, not a payment gateway.

3 Actors and the project journey

Client

A business owner who needs a service. Creates a project, picks one or more services, writes customizations, asks for a meeting when needed, and follows progress and support. Deals with Al-Alamia only.

Freelancer

A skilled professional. Registers, chooses a track, adds work samples and waits for admin approval. Once approved, receives task offers that match the track and skills, accepts or declines, delivers, and follows earnings and deductions.

Admin

Runs the workflow: approves freelancers, reviews and prices requests, creates tasks, reviews deliveries, and applies deductions and blocks. Not a separate table; a role in Identity.

A project from start to finish



A worked example

A restaurant owner asks for a website. The admin reviews the request and sets the final price, then creates three tasks: interface design (UI/UX), API development (Backend) and front-end integration with testing (Frontend). Each task is offered to the approved freelancers in that track. The first to accept gets it and the other offers close. On delivery the admin reviews: accept, ask for a revision with a new deadline, or reject for good, which sends the task back for reassignment. In every case the financial outcome is recorded.

> **A revision is not a new task.** A revision request stays attached to the original task and stays with the same freelancer under a new deadline. That keeps responsibility clear and stops the task count from ballooning.

4 Business rules

These rules are the contract between product and code. Each has an ID that is used in tests and pull requests. Numeric values come from configuration (`PolicyOptions`), not constants.

ID	Rule	Enforced in
BR-01	A freelancer receives no offers until the admin approves their account (`Approved`).	Eligible-freelancer query
BR-02	Clients deal with Al-Alamia only; there is no direct contact between client and freelancer.	API and UI design
BR-03	The admin prices the project and creates its tasks by hand.	Role policies
BR-04	A task belongs to one freelancer at a time, and the first valid accept decides who.	Conditional update + `RowVersion`
BR-05	Once a task is assigned, the remaining offers on it are closed.	Accept transaction
BR-06	At most 10 accepted tasks per freelancer per calendar month. A task counts the moment it is accepted, even if the freelancer later withdraws.	Accept transaction
BR-07	The announced deadline is `PublishedDeadline` . If it is missed, one extra day (`ExtensionDeadline`) is granted with a 20% deduction from the task reward.	Deadline service
BR-08	The 20% deduction applies once per task, whatever the reason.	Filtered unique index on the ledger
BR-09	No delivery even after the extra day: a 30-day block.	Deadline background job
BR-10	Final rejection of a delivery: the task is reassigned and the freelancer is blocked for 7 days.	Delivery review
BR-11	Withdrawing after accepting: 7-day block and reassignment.	Withdrawal service
BR-12	A block only stops new offers. Tasks already assigned are not cancelled; the freelancer finishes them during or after the block.	Eligible query + accept
BR-13	A revision stays on the original task with the same freelancer under a new deadline; no new task is created.	`TaskRevision`
BR-14	Every published portfolio item has exactly 3 images.	Publish service + validation
BR-15	The platform records earnings and deductions only; no real money moves.	`FreelancerLedgerEntry`
BR-16	The marketing package lasts 6 months, and a subscriber keeps a copy of the items and price at purchase time. Editing the original package never changes existing subscriptions.	Snapshot tables
BR-17	A meeting request is optional (for customizations), the proposed slot is today or tomorrow, and the admin approves or rejects with a notification to the client.	`Meeting`
BR-18	A freelancer has one primary track; a skill may belong to more than one track.	Link tables

BR-19	One `File` table, with link tables to projects, tasks, submissions, portfolio items and support messages.	File layer
BR-20	Roles come from Identity (a user may hold several). There is no Admin table.	Identity

Default policy values

Configuration key	Value	Rule
`Policy:MaxAcceptedTasksPerMonth`	10	BR-06
`Policy:LateGraceDays`	1	BR-07
`Policy:LateDeductionPercent`	20	BR-07, BR-08
`Policy:BlockDaysLateNoShow`	30	BR-09
`Policy:BlockDaysFinalRejection / Withdrawal`	7 / 7	BR-10, BR-11
`Policy:PortfolioImagesRequired`	3	BR-14
`Policy:MarketingPackageMonths`	6	BR-16

One point for the team to confirm. While gathering requirements the monthly cap was mentioned with two different values (5 and 10). This document uses **10**, the figure in the original idea document, and it is now a single setting that can change at any time. A "calendar month" is calculated in Cairo time, and dates are stored in UTC.

5 System architecture

5.1 The big picture

A two-tier web application: a React front end talks over HTTPS to a single Web API, and that API is the only door to the SQL Server database. We chose a **modular monolith** on purpose. One deployable unit stays simple to run and trace in a five-week build, while clear module boundaries leave the door open to splitting it later if that ever becomes necessary.

React SPA	Vite · TypeScript · React Router · TanStack Query · react-i18next (RTL/LTR) · CSS-variable design system
alealamia.API	Thin controllers · authorization · rate limiting · ProblemDetails errors · OpenAPI
alealamia.Application	Use cases · validation (FluentValidation) · DTOs · repository and service interfaces · policies
alealamia.Domain	Entities · enums · state rules (valid transitions only) · no framework dependencies
alealamia.Infrastructure	EF Core 10 + SQL Server · Identity · JWT · file storage · e-mail (MailKit) · background jobs

Dependencies point inward only: `Api → Application → Domain`, while `Infrastructure` implements the interfaces declared in `Application`. That lets us test business rules without a database or HTTP.

5.2 Solution structure

```
alealamia/
├─ src/
│  ├─ alealamia.Domain/            # entities, enums, domain rules
│  ├─ alealamia.Application/       # use cases, DTOs, validators, abstractions
│  │   └─ Features/             # Identity, Catalog, Projects, Tasks, Delivery, Finance,
Marketing, Support, Notifications
│  ├─ alealamia.Infrastructure/    # EF Core, migrations, identity, files, email, background
jobs
│  └─ alealamia.API/               # controllers, middleware, DI, OpenAPI
├─ tests/
│  ├─ alealamia.Domain.Tests/      # fast unit tests of rules BR-xx
│  ├─ alealamia.Application.Tests/
│  └─ alealamia.API.IntegrationTests/  # Testcontainers (SQL Server), concurrency tests
├─ web/                         # React app (Vite)
├─ docs/                        # this document, ERD, ADRs, screenshots
└─ docker-compose.yml
```

The Application layer is organized by feature (one folder per business module) rather than by type, so each member works inside their own module without file conflicts.

5.3 Technical decisions and why

Decision	Choice	Reason
Style	Clean Architecture inside a modular monolith	Keeps rules apart from the framework, tests fast, and deployment simple
Data access	EF Core 10 with `AsNoTracking` and projections for reads	High productivity, with raw control of the SQL on critical paths (`ExecuteUpdate`)
Identity	ASP.NET Core Identity + short-lived JWT + rotating refresh tokens	Roles out of the box; only the `TokenHash` is stored, never the token
Cross-module calls	Direct calls to Application interfaces	No message broker: less complexity and no paid library dependency
Validation	FluentValidation at the use-case boundary	Uniform, translatable error messages
Errors	`ProblemDetails` (RFC 9457) with a stable business code	The UI translates the code and shows the right Arabic or English message
Background work	`BackgroundService` (deadline monitor, e-mail sender)	No external dependency; `EmailNotificationLog` acts as an outbox
Files	`IFileStorage` interface, local-disk implementation for now	Cloud storage can replace it later without touching business logic
API docs	Built-in .NET 10 OpenAPI + Scalar UI	A live API contract the front-end team can rely on
Logging	Serilog (structured) + correlation id	Follow one request from the UI down to the database
Front end	React + TanStack Query	Smart caching and refetching cut requests and simplify state

5.4 Files, e-mail and notifications

- **Files.** One upload endpoint validates the type, the size (10 MB maximum) and the extension against the content, then creates a `File` record linked to the right entity. Downloads go through an authorized endpoint; there is no open public link.
- **Notifications.** A `Notification` record is shown in the app and polled every 30 seconds in this release. It carries `RelatedEntityType/Id` so a click opens the relevant page directly.
- **E-mail.** An `EmailNotificationLog` row with a "pending" status is written inside the same transaction as the event. A background job sends it, updates the status and records any error. A failed send never fails the original operation.

6 State machines

The current state lives in a `Status` column on the entity, and the change history lives in a separate table (`ProjectStatusHistory`, `TaskStatusHistory`). Valid transitions are defined in the Domain layer; any other attempt raises a business exception that surfaces as `409` or `422`.

6.1 Project status

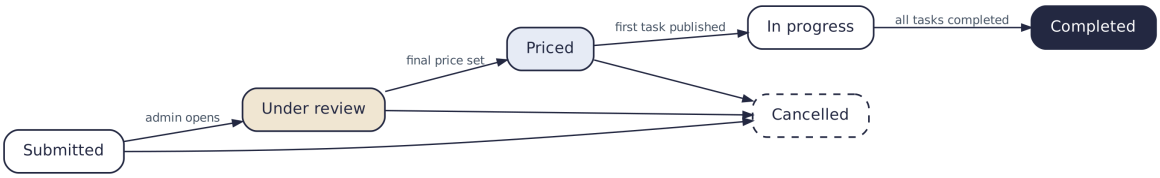


Figure 1 · Project lifecycle

6.2 Task status

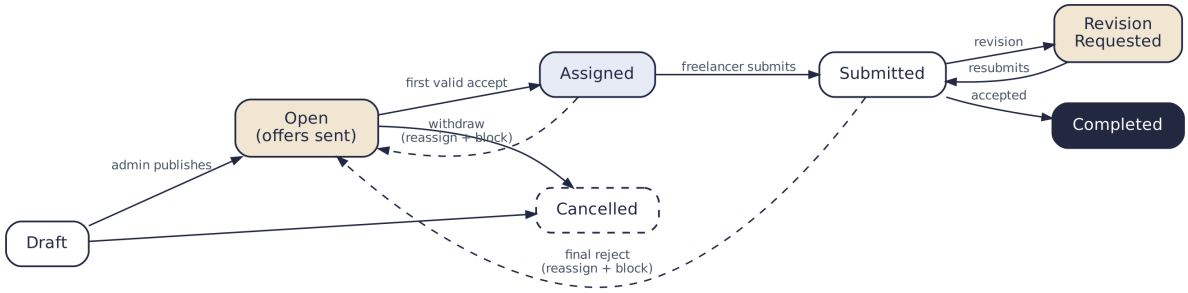


Figure 2 · Task lifecycle. The two dashed lines are the reassignment paths (withdrawal and final rejection); both apply a block.

6.3 Other entities

Entity	States	Note
FreelancerProfile.ApprovalStatus	Pending → Approved | Rejected	A rejected profile can reapply after changes
TaskOffer	Pending → Accepted | Declined | Closed	`Closed` means another freelancer won
TaskRevision	Open → Completed	At most one open revision per task
Meeting	Pending → Approved | Rejected	The client is notified either way
ProjectMarketingPackage	Active → Expired | Cancelled	Expires automatically at `EndDate`
SupportTicket	Open → InProgress → Closed	Closing sets `ClosedAt`

7 Critical flows

These three flows are what keep the platform correct under load, so they are written and tested first and with the most care.

7.1 First accept wins

The problem: two freelancers press "Accept" at the same instant. The fix is to make the decision a **single atomic conditional update** on the task row, rather than a check followed by a write. Whoever gets "one row affected" wins; everyone else receives `409`.

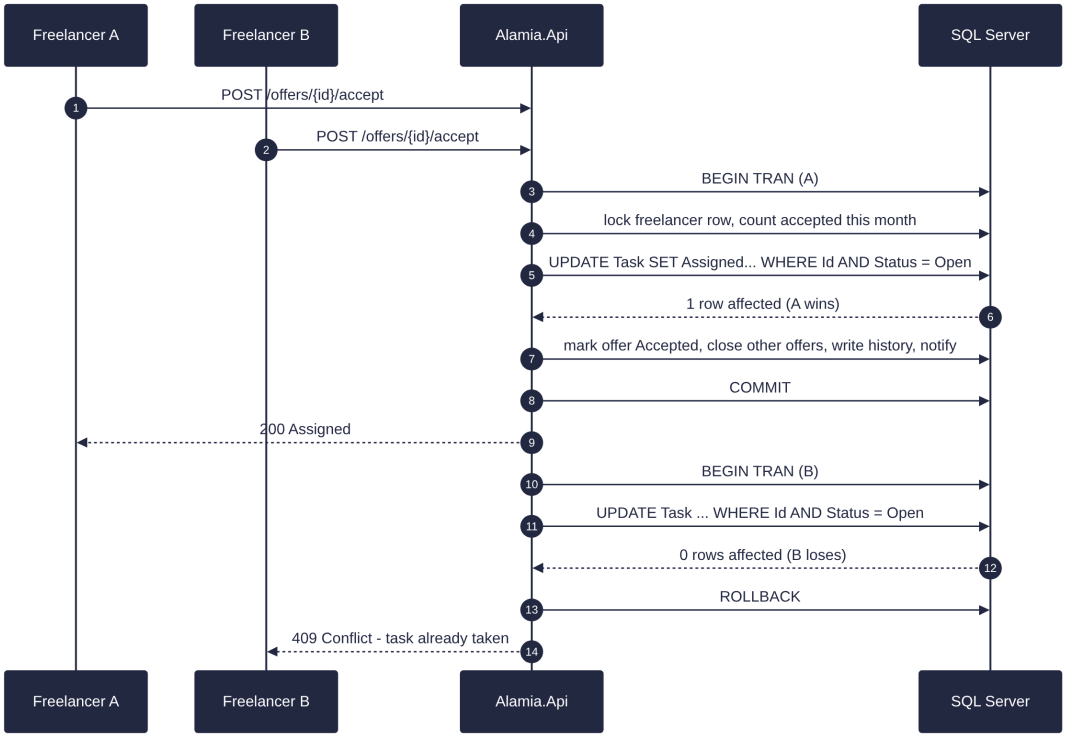


Figure 3 · Concurrent accept sequence

```
// Application: AcceptOfferHandler (simplified)
await using var tx = await db.Database.BeginTransactionAsync(ct);

// 1) Serialize this freelancer's accepts so the monthly count is reliable
await db.FreelancerProfiles.Where(f => f.Id == me)
        .ExecuteUpdateAsync(s => s.SetProperty(f => f.UpdatedAt, now), ct);

var accepted = await db.TaskOffers.CountAsync(o => o.FreelancerProfileId == me
        && o.Status == OfferStatus.Accepted
        && o.RespondedAt >= monthStartUtc && o.RespondedAt < nextMonthStartUtc, ct);
if (accepted >= policy.MaxAcceptedTasksPerMonth) return Error.MonthlyCapReached;

// 2) The decider: one atomic conditional update
var rows = await db.Tasks
    .Where(t => t.Id == taskId && t.Status == TaskStatus.Open && t.AssignedFreelancerProfileId
== null)
    .ExecuteUpdateAsync(s => s
        .SetProperty(t => t.Status, TaskStatus.Assigned)
        .SetProperty(t => t.AssignedFreelancerProfileId, me), ct);
if (rows == 0) { await tx.RollbackAsync(ct); return Error.TaskAlreadyTaken; }   // 409

// 3) Everything else happens in the same transaction
offer.MarkAccepted(now); await CloseOtherOffers(taskId, ct);
history.Add(taskId, Open, Assigned, me); notifications.Enqueue(...);
await db.SaveChangesAsync(ct); await tx.CommitAsync(ct);
```

A `RowVersion` on the task is a second line of defence against any other concurrent edit, such as an admin changing the task during an accept. A unique index on `TaskOffer(TaskId, FreelancerProfileId)` stops the same person from receiving the same offer twice.

> **How we prove it works.** An integration test fires 50 parallel accept requests at one task through Testcontainers and asserts **exactly one winner**, 49 `409` responses, and a task assigned to a single freelancer. Passing this test is a condition for closing week 3.

7.2 The monthly cap

The count comes from `Accepted` offers whose `RespondedAt` falls inside the calendar month (Cairo time converted to UTC). Because an offer stays `Accepted` even after the freelancer withdraws, a task counts "at the moment of acceptance", exactly as BR-06 says, with no extra column. The index `(FreelancerProfileId, Status, RespondedAt)` makes the count a pure index operation.

7.3 Deadlines, deductions and blocks

Event	Check	Effect (one transaction)
Delivery after `PublishedDeadline` and before `ExtensionDeadline`	Once per task	`LateDeduction` entry = 20% of `RewardAmount`; freelancer notified
`ExtensionDeadline` passes with no delivery	Deadline monitor, every 5 minutes	30-day block, task returned for assignment, history row, notification
Delivery accepted	Admin decision	`Earning` entry = `RewardAmount`; task `Completed`
Final rejection	Admin decision	7-day block, `TaskReassignment`, task back to `Open`
Withdrawal after accepting	Freelancer request	7-day block, `TaskReassignment`, task back to `Open`

The filtered unique index `(TaskId, EntryType) WHERE EntryType = LateDeduction` makes the "one deduction per task" rule (BR-08) a database guarantee, even if the monitor restarts or an event fires twice.

A block is "active" while `EndsAt > now`. The check goes through the `(FreelancerProfileId, EndsAt)` index and is used in only two places: choosing who receives offers, and validating at accept time.

8 Database design and ERD changes

The design has 40 entities in 12 modules, in a single database. We started from the ERD the team had drawn and reviewed it against the idea and the business rules. **No features were added**; every change closes a gap that the agreed rules need in order to be implemented precisely.

8.1 ERD review: what changed and why

#	Change	Reason
1	New `Task.RewardAmount` (decimal 18,2)	The 20% deduction and the earning need a base amount. Without it, `FreelancerLedgerEntry.Amount` cannot be derived from any other field.
2	New `Task.RowVersion` (rowversion)	A concurrency token that protects assignment and any other simultaneous edit of a task.
3	`AssignedFreelancerId` renamed to `AssignedFreelancerProfileId`	Naming consistency: the foreign key points to `FreelancerProfile`, as in the other tables.
4	`FreelancerProfile.ProfileImageUrl` replaced by `ProfileImageFileId` (optional FK to `File`)	Keeps the "one file table" rule (BR-19). A text URL breaks it and makes access control harder.
5	Optional `Notification.RelatedEntityType / RelatedEntityId`	So a notification opens the right page (task, project, ticket) without guessing from its text.
6	New `RefreshToken` table	Token rotation and revocation at logout. Security plumbing rather than a feature; only `TokenHash` is stored.
7	Explicit constraints and indexes (8.3)	An ERD cannot express unique and filtered indexes, and rules BR-04, BR-08 and BR-14 depend on them.

What we reviewed and kept as it was

- **`Meeting` stays linked to `Project`**, not to a single service: a meeting is about the customizations of the whole project. Its `ClientProfileId` can be derived from the project but is kept to speed up the client's meeting list, and the service checks that the two agree.
- **History tables stay separate from current state**, so reading the current state never touches the history table.
- **Snapshots in packages** (`NameSnapshot`, `PriceAtPurchase`) deliver BR-16 as drawn.
- **No assessment tables** (Assessments), on purpose: they came up in earlier discussions, are outside the agreed scope, and move to future work.
- **No per-service price in `Project.FinalPrice`**: the price is per project, as the idea says (the admin sets the final project price), while tasks carry the freelancer reward separately.

> **A note on pricing.** `Project.FinalPrice` is what the client pays and `Task.RewardAmount` is what the freelancer earns. The difference is Al-Alamia's margin, an administrative decision the admin enters; the platform does not calculate it automatically in this version.

8.2 The full schema by module

The following pages show the twelve modules in ten diagrams. Grey italic boxes are entities from other modules, shown only to indicate the relationship. PK is a primary key, FK a foreign key, and ? marks an optional field. The source file `alamia-erd.mmd` (Mermaid) ships with the package.

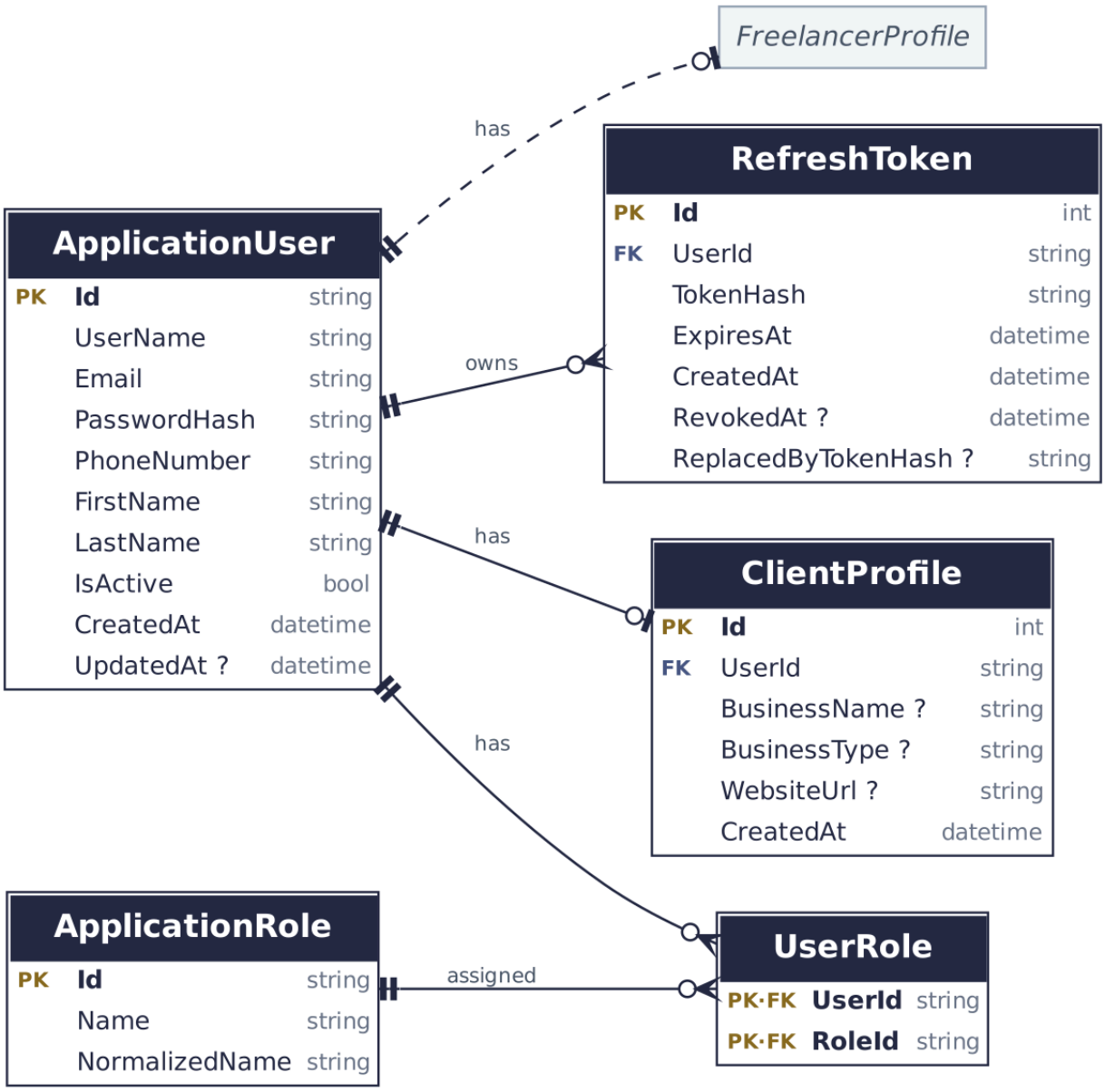


Figure 4 · Identity and users. `ApplicationUser` is referenced by fields such as `CreatedByUserId` and `ChangedByUserId` in other modules; those lines are left out here to avoid clutter.

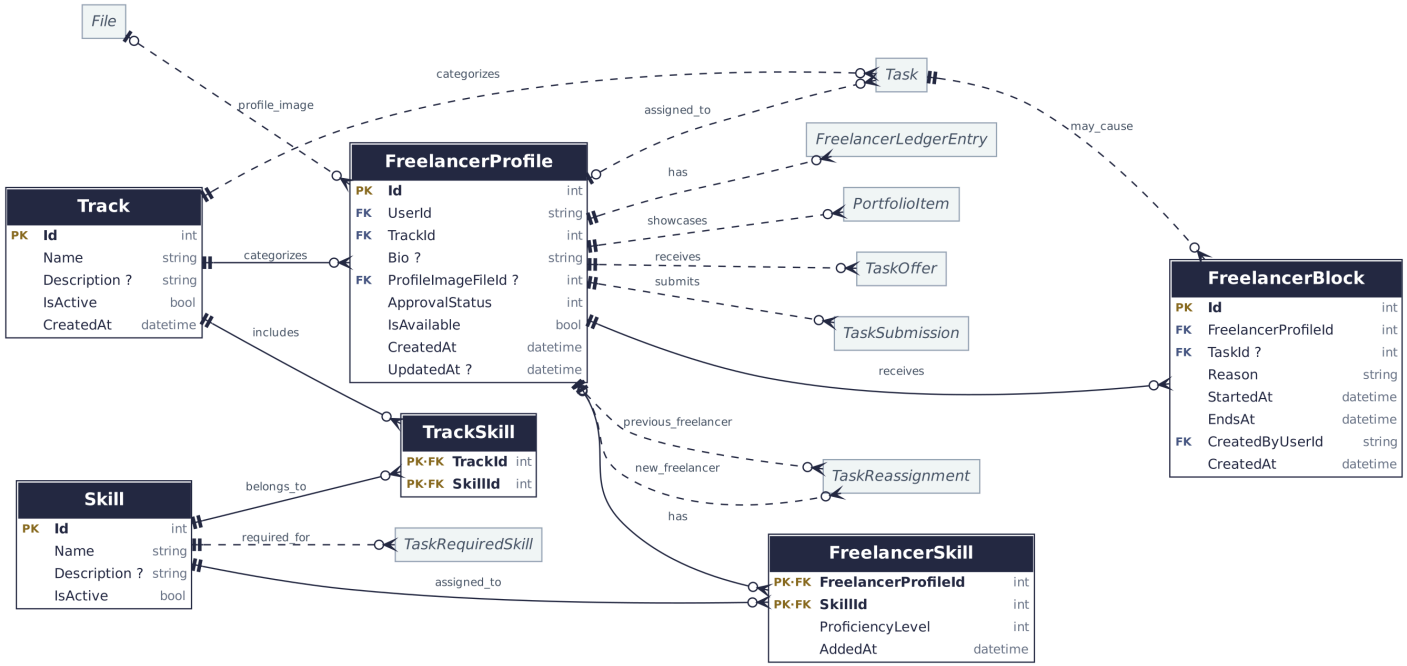


Figure 5 · Freelancer: tracks, skills and blocks

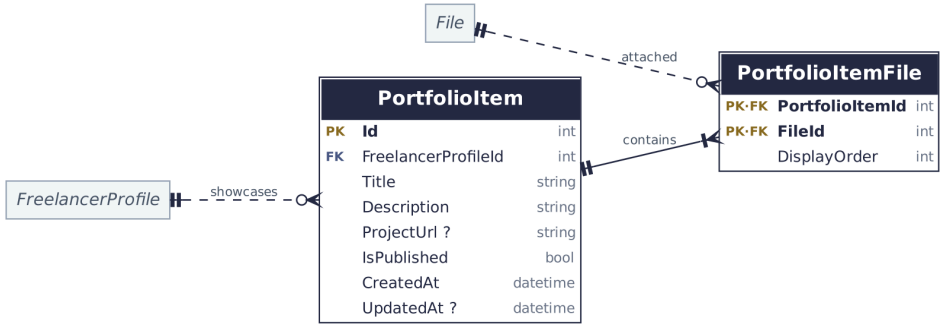


Figure 6 · Portfolio

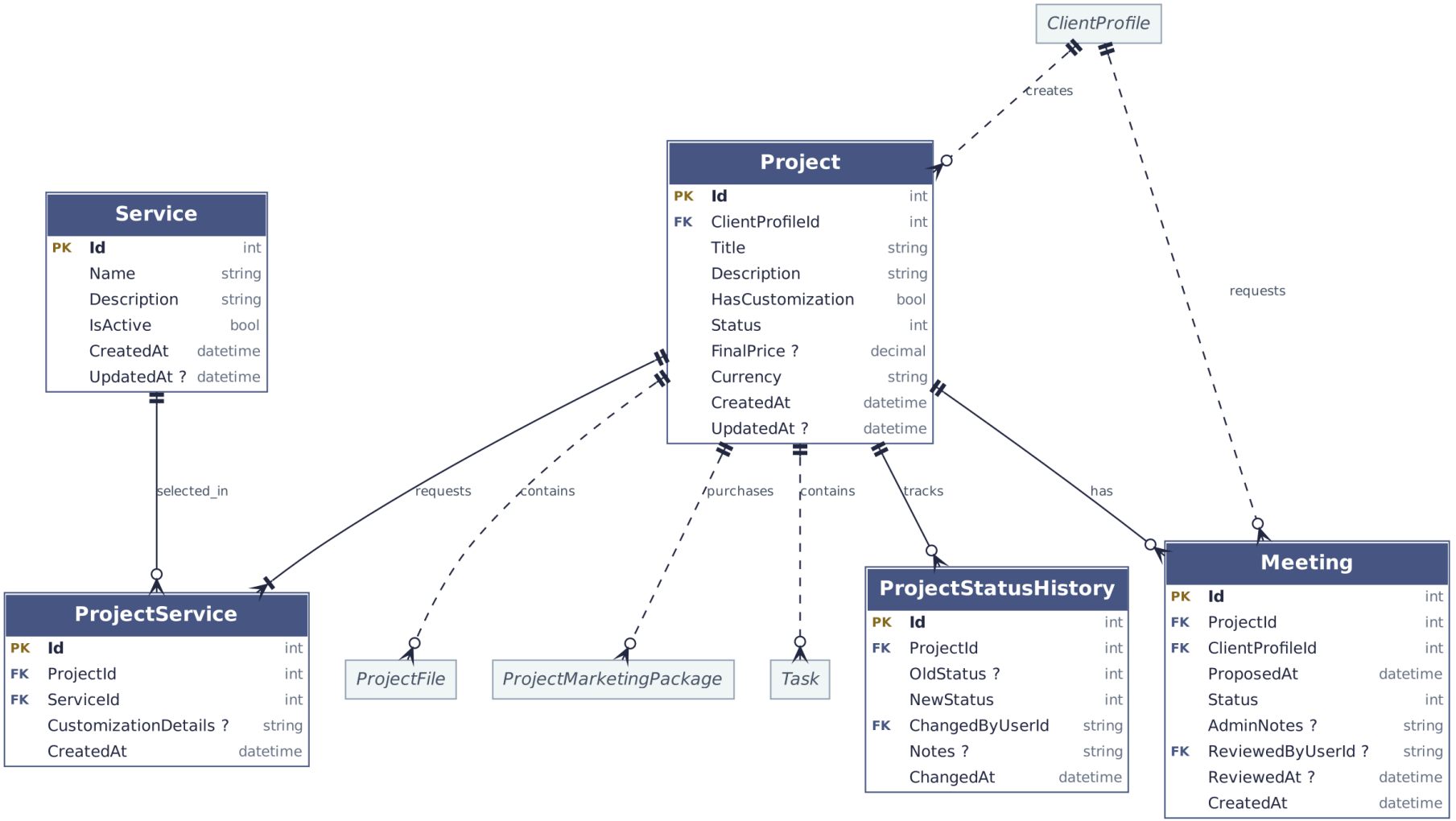


Figure 7 · Services, projects and meetings

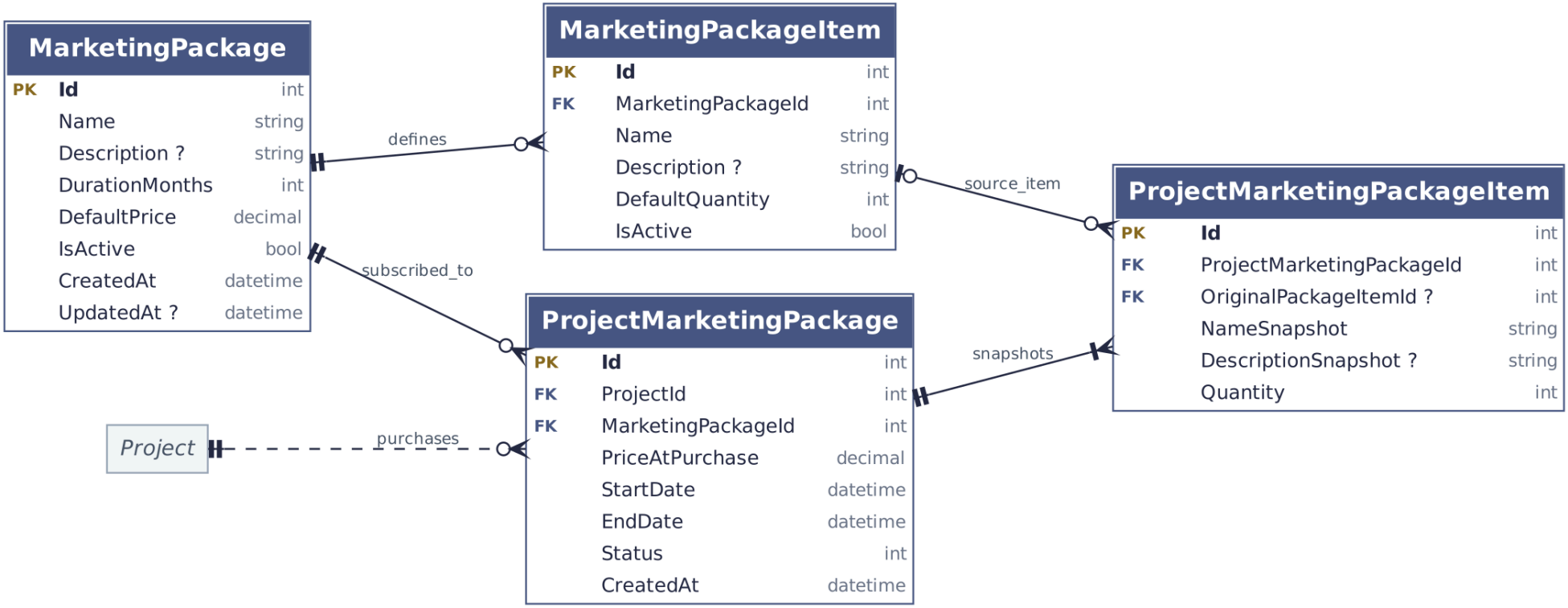


Figure 8 · Marketing packages (snapshot at purchase)

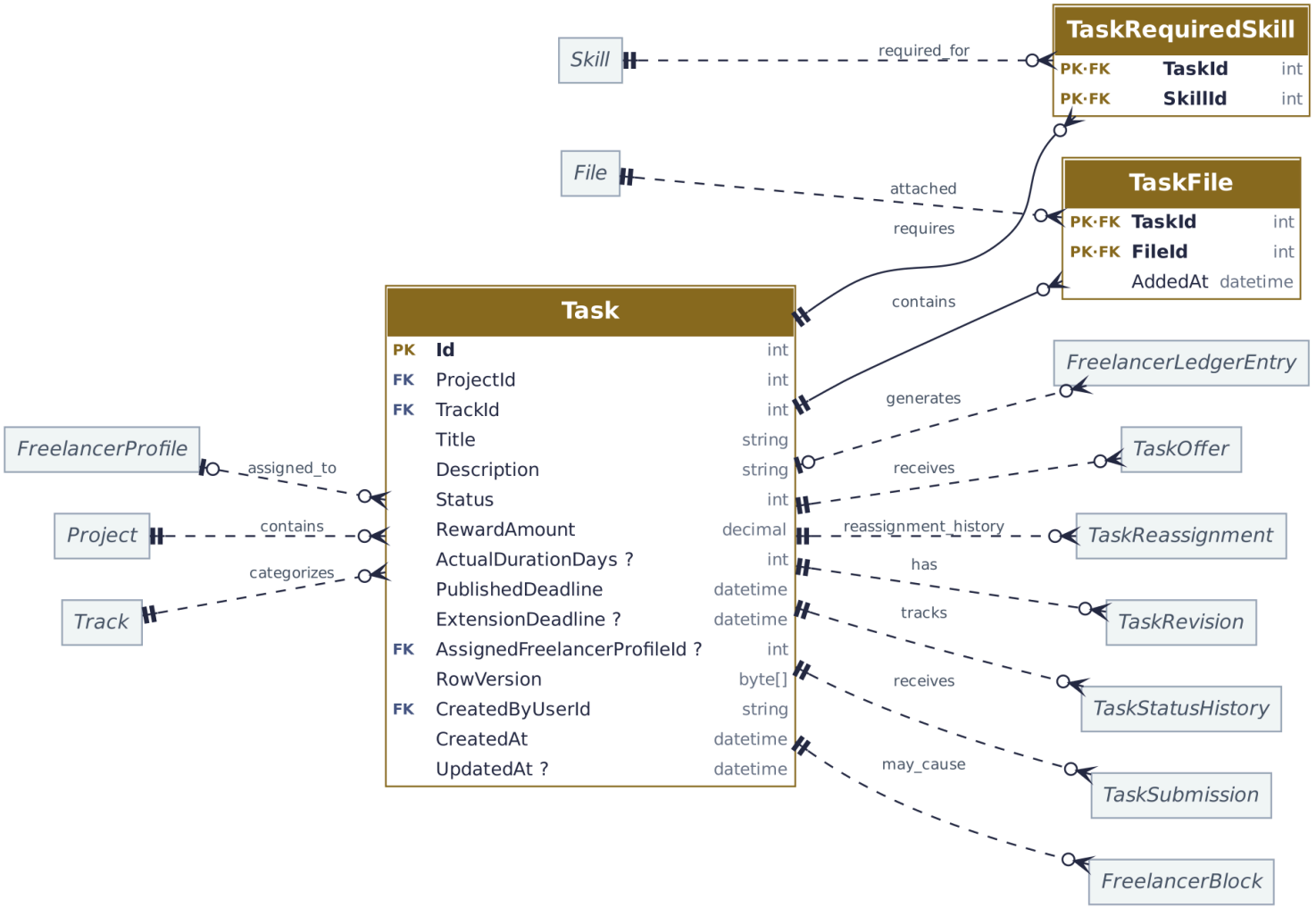


Figure 9 · Task core

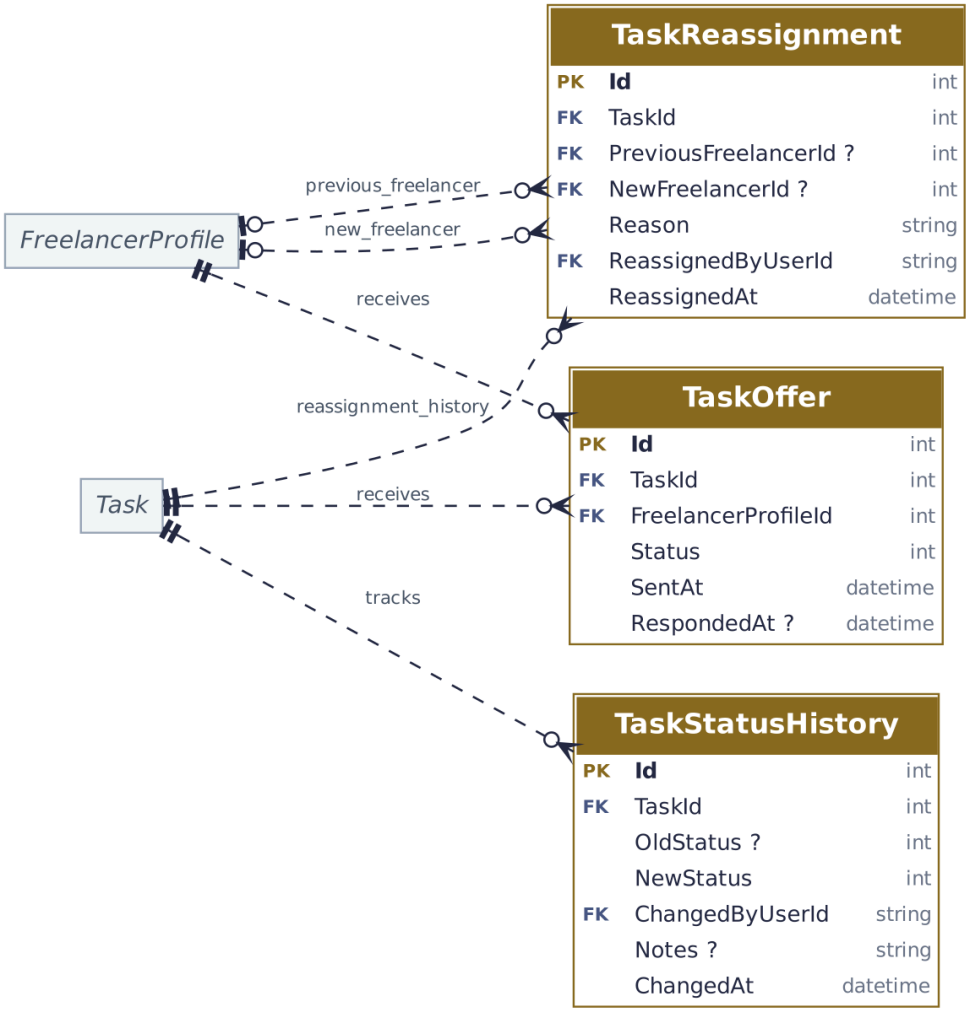


Figure 10 · Task offers, history and reassignment

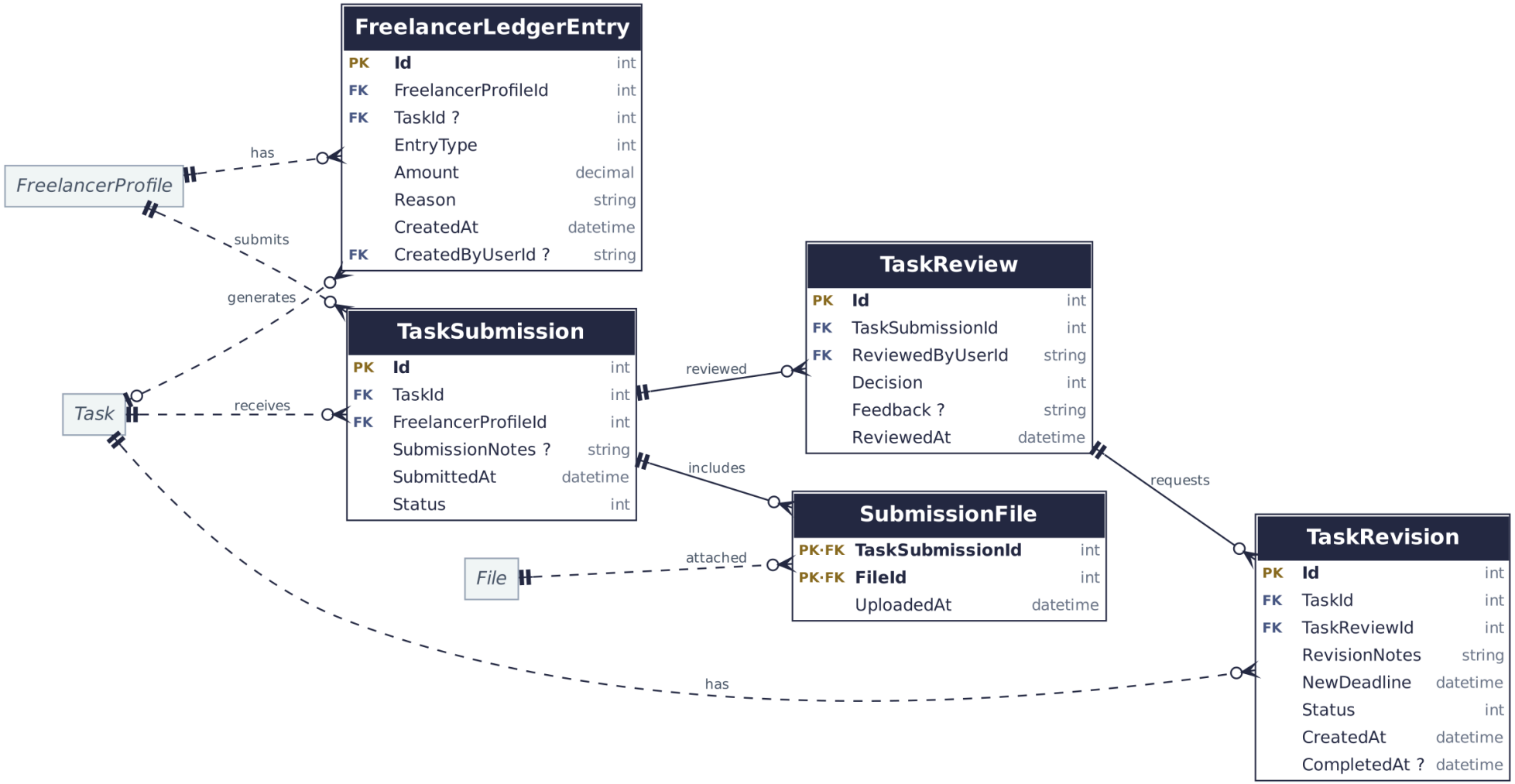


Figure 11 · Delivery, review and the financial ledger

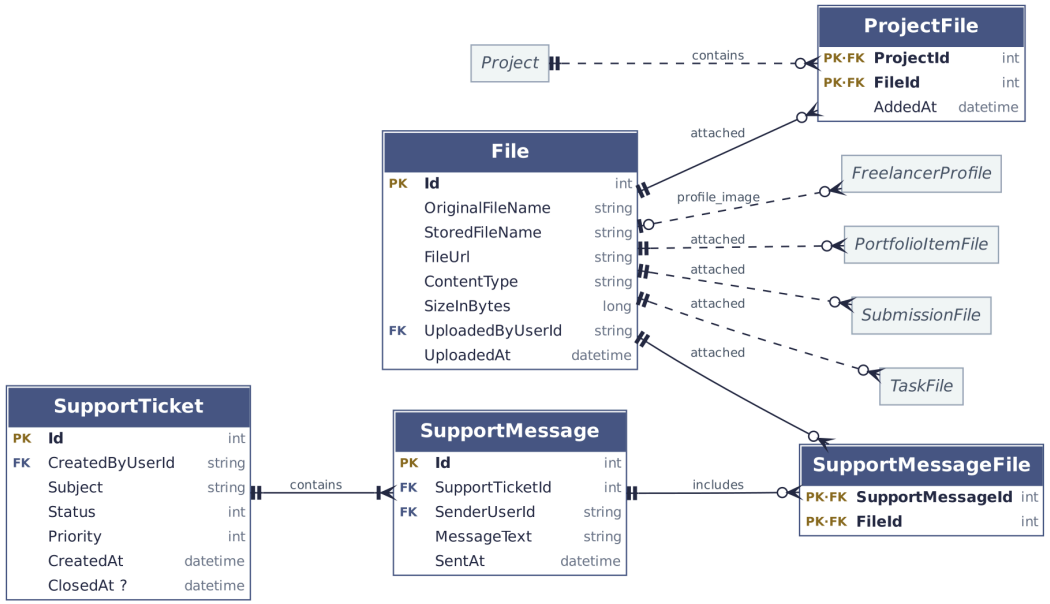


Figure 12 · Files and support

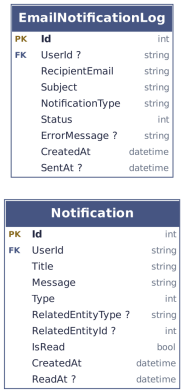


Figure 13 · Notifications

8.3 Constraints and indexes

Every index here is designed around a real query on the platform, not added at random, and each one maps to a specific path.

Table	Index / constraint	Serves
ClientProfile · FreelancerProfile	Unique on `UserId`	One profile per user
FreelancerProfile	`(TrackId, ApprovalStatus, IsAvailable)`	Choosing eligible freelancers for an offer
FreelancerBlock	`(FreelancerProfileId, EndsAt)`	Active-block check
FreelancerSkill	Composite PK + index `(SkillId)`	Skill matching
PortfolioItemFile	Composite PK + unique `(PortfolioItemId, DisplayOrder)`	Image order (exactly 3 enforced on publish)
Project	`(ClientProfileId, Status, CreatedAt DESC)`	The client's project list
Task	`(Status, TrackId, PublishedDeadline)` and `(AssignedFreelancerProfileId, Status)`	Admin board, "my tasks", deadline monitor
TaskOffer	Unique `(TaskId, FreelancerProfileId)`; `(FreelancerProfileId, Status, SentAt DESC)`; `(FreelancerProfileId, Status, RespondedAt)`	No duplicate offers, the offer feed, the monthly count
FreelancerLedgerEntry	`(FreelancerProfileId, CreatedAt DESC)`; filtered unique `(TaskId, EntryType) WHERE EntryType = 2`	The freelancer's ledger, the one-deduction rule
Notification	`(UserId, IsRead, CreatedAt DESC)`	Unread counter and list
EmailNotificationLog	`(Status, CreatedAt)`	The outbox
SupportTicket · SupportMessage	`(CreatedByUserId, Status)` and `(SupportTicketId, SentAt)`	Ticket list and message thread
RefreshToken	Unique on `TokenHash`; index `(UserId, RevokedAt)`	Token rotation and revocation

General conventions

- Amounts are `decimal(18,2)` in a single currency (a text `Currency` on the project). Dates are UTC.
- Enums are stored as integers with a conversion in EF; their values are in Appendix A.
- Deletes: entities with history (projects, tasks, ledger entries) are never physically deleted. Where it makes sense they are deactivated through `IsActive`.

9 API design

9.1 Conventions

- REST over JSON, base path `/api/v1`. Resource names are plural and in English.
- Authentication: `Authorization: Bearer`. Access tokens live 15 minutes; refresh tokens live 14 days and rotate on every use.
- Authorization uses role-based policies, with ownership checked inside the use case (a client sees only their own projects).
- Pagination: **keyset pagination** for growing lists (tasks, notifications, ledger) and `page/pageSize` for small admin-managed lists.
- Errors: `ProblemDetails` with a stable `code` such as `task.already_taken`, and an `errors` map for fields on 400/422.
- Status codes: 200/201/204 for success; 400 validation; 401; 403; 404; **409 for conflicts (concurrent accept)**; 422 business rule; 429 rate limit.
- Rate limiting: a stricter policy on `/auth/*` and file upload.

9.2 Endpoint groups

Module	Main routes	Role
Auth	`POST /auth/register · /auth/login · /auth/refresh · /auth/logout · /auth/forgot-password · /auth/reset-password`	Public
Profiles	`GET/PUT /me · GET/PUT /freelancers/me · GET/POST/PUT/DELETE /freelancers/me/portfolio · POST …/portfolio/{id}/publish`	Client / freelancer
Catalog	`GET /tracks · /skills · /services (cached) · CRUD under /admin/tracks|skills|services`	All / admin
Freelancer admin	`GET /admin/freelancers?status= · POST /admin/freelancers/{id}/approve|reject · POST …/block`	Admin
Projects	`POST /projects · GET /projects · GET /projects/{id} · POST /projects/{id}/meetings · POST /admin/projects/{id}/price · POST …/status`	Client / admin
Meetings	`GET /admin/meetings · POST /admin/meetings/{id}/approve|reject`	Admin
Tasks (admin)	`POST /admin/projects/{id}/tasks · PUT /admin/tasks/{id} · POST /admin/tasks/{id}/publish · POST …/reassign · GET /admin/tasks?status=&trackId=`	Admin
Offers	`GET /offers (freelancer feed) · POST /offers/{id}/accept · POST /offers/{id}/decline · POST /tasks/{id}/withdraw`	Freelancer
Delivery	`POST /tasks/{id}/submissions · GET /admin/submissions?status= · POST /admin/submissions/{id}/review (accept | revision | reject)`	Freelancer / admin

Finance	`GET /freelancers/me/ledger` · `GET /admin/ledger` · `POST /admin/ledger/adjustments`	Freelancer / admin
Marketing	`GET /marketing/packages` · `POST /projects/{id}/marketing` · `GET/PUT /admin/marketing/packages`	Client / admin
Files	`POST /files` · `GET /files/{id}`	Any user
Support	`POST /support/tickets` · `GET /support/tickets` · `POST /support/tickets/{id}/messages` · `POST …/close`	All
Notifications	`GET /notifications` · `GET /notifications/unread-count` · `POST /notifications/{id}/read` · `POST /notifications/read-all`	All

> **Contract first.** At the end of week 1 the OpenAPI file for each module is frozen before any logic is written. The front-end teammate builds against a stable contract, and any change to it becomes an explicit conversation instead of a surprise.

10 UI design and brand

10.1 The identity: "the gate"

The logo is a single gate sheltering two people. **The gold figure is the client** and **the light-blue figure is the freelancer**. They stand side by side at the same height under the navy frame of Al-Alamia: the two sides are equals, and the platform is the roof that brings them together. The heavy, upright Arabic wordmark gives a sense of trust and stability, paired with the Latin name `AL-ALAMIA` in wide letter spacing for international audiences.



Usage rule. Gold and light blue have weak contrast on light backgrounds, so they are never used for text or buttons there. Primary buttons are navy `#242944` (client and admin) or slate `#485982` (freelancer) with white text; gold and light blue serve as highlights, selection backgrounds and tags. On a navy background the frame and wordmark turn cream. The approved versions are: primary on light, dark, and single-colour (for stamps and print).

10.2 Design tokens

Category	Values
Colours	`--navy #242944 · --slate #485982 · --gold #C9A15F · --sky #B9CBEA · --cream #F6F2EF · --line #E4DED5 · --danger #B42318`
Typeface	IBM Plex Sans Arabic (free and open source), weights 400/500/600/700, for Arabic and Latin together
Type scale	`12 · 14 · 15 · 18 · 22 · 28 · 36 px`
Spacing and radius	`4 · 8 · 12 · 16 · 24 · 32 px · radius 12 (fields, buttons) / 18 (cards)`
Touch and access	Minimum button and field height 44–48 px, visible focus ring, text contrast ≥ 4.5:1

10.3 UI principles

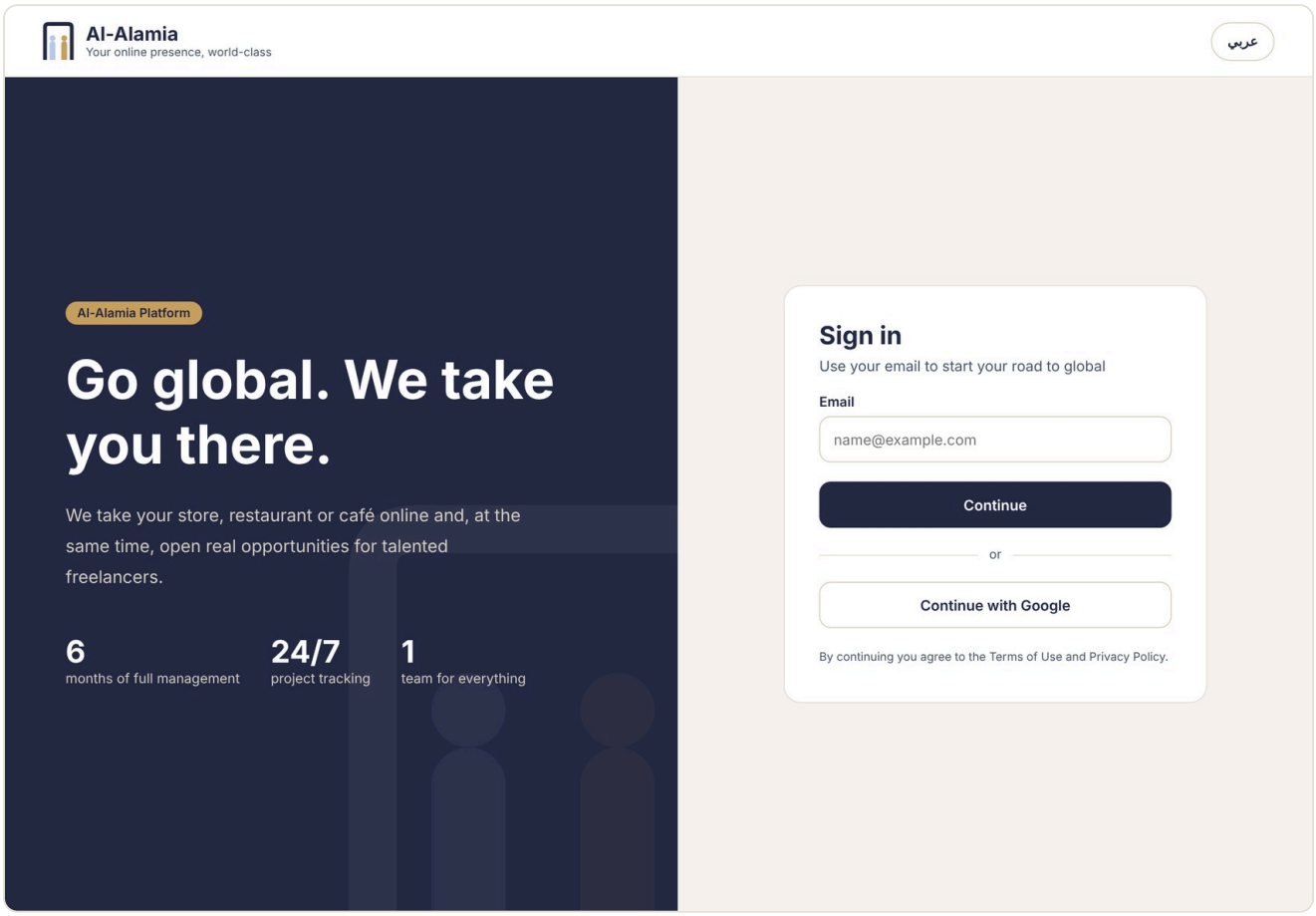
- **RTL first.** The UI uses logical properties (`margin-inline`, `padding-inline`), so switching to LTR needs no second copy.
- **Three app areas with separate routes** inside one React project, lazy-loaded with `React.lazy`: `/app` for the client, `/work` for the freelancer, `/admin` for the admin.
- **Server state in TanStack Query** with consistent keys, and local UI state kept local. No Redux.
- **Four states per screen:** loading, empty, error, success. No blank screens.
- **Forms** use React Hook Form + Zod, and error messages come from the translated business code.

10.4 The prototype in the final brand

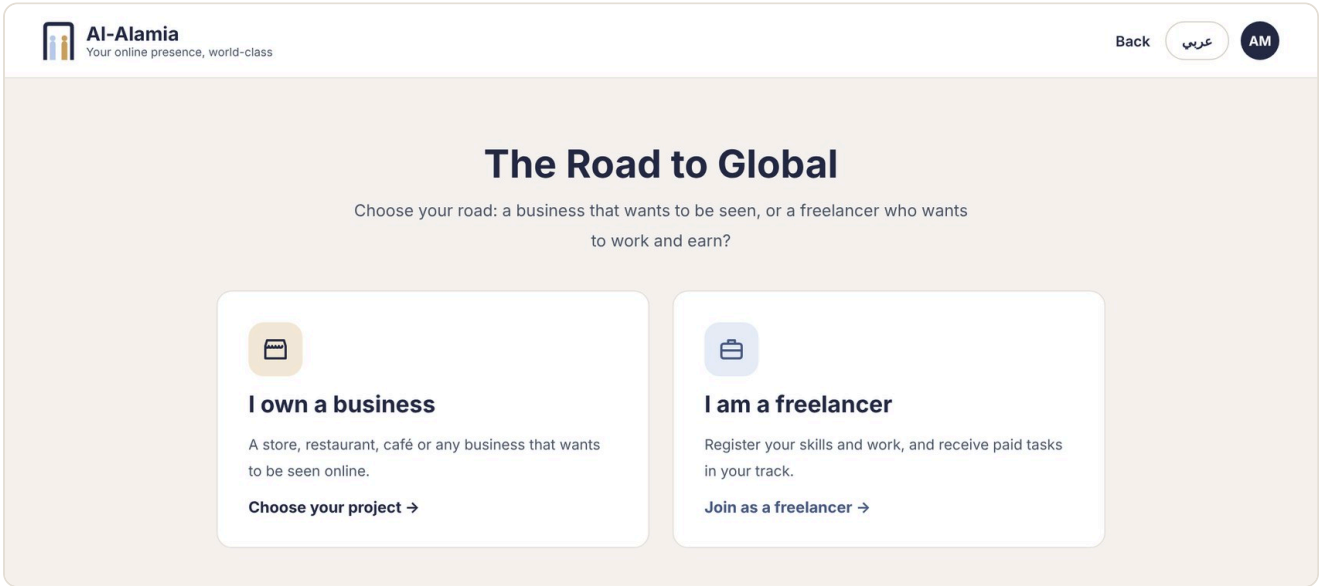
The screenshots below come from the interactive prototype built to explain the idea. It is a reference for experience and layout, not the final code: it will be rebuilt in React on top of the real API, keeping the brand and the flow.

> **Prototype versus implementation.** Numbers and names in the images are sample data. The "Payouts" screen shows the ledger and does not execute a transfer. Details such as the monthly task cap are read from policy configuration, not from the UI.

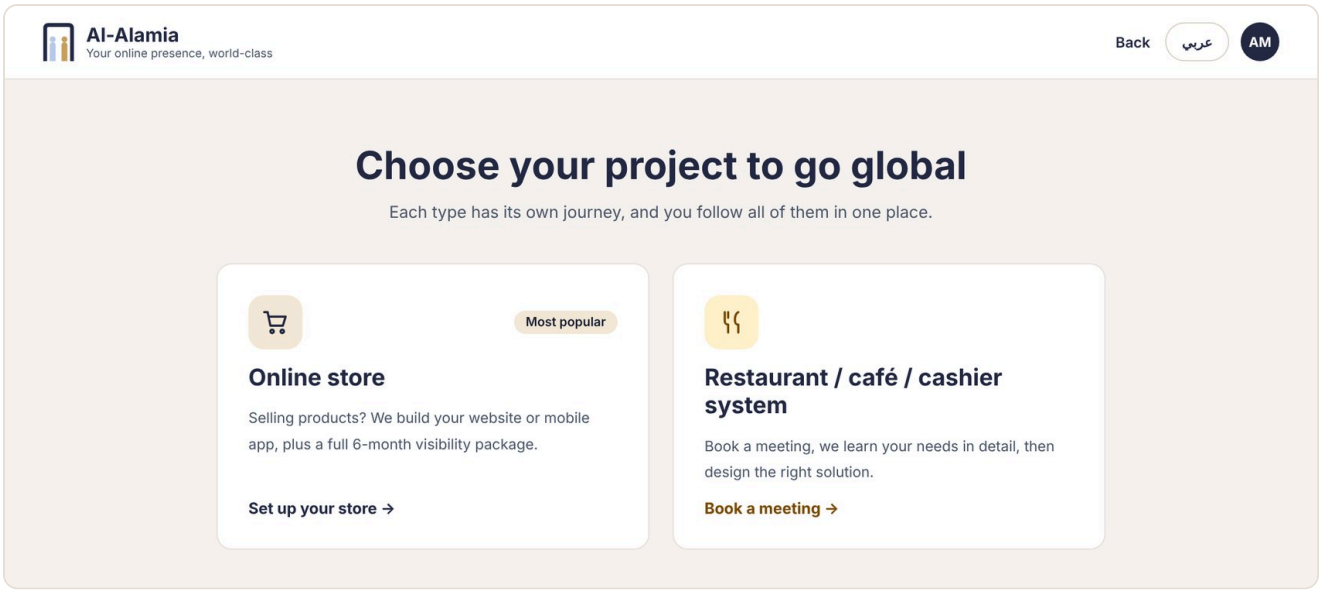
A) Client journey



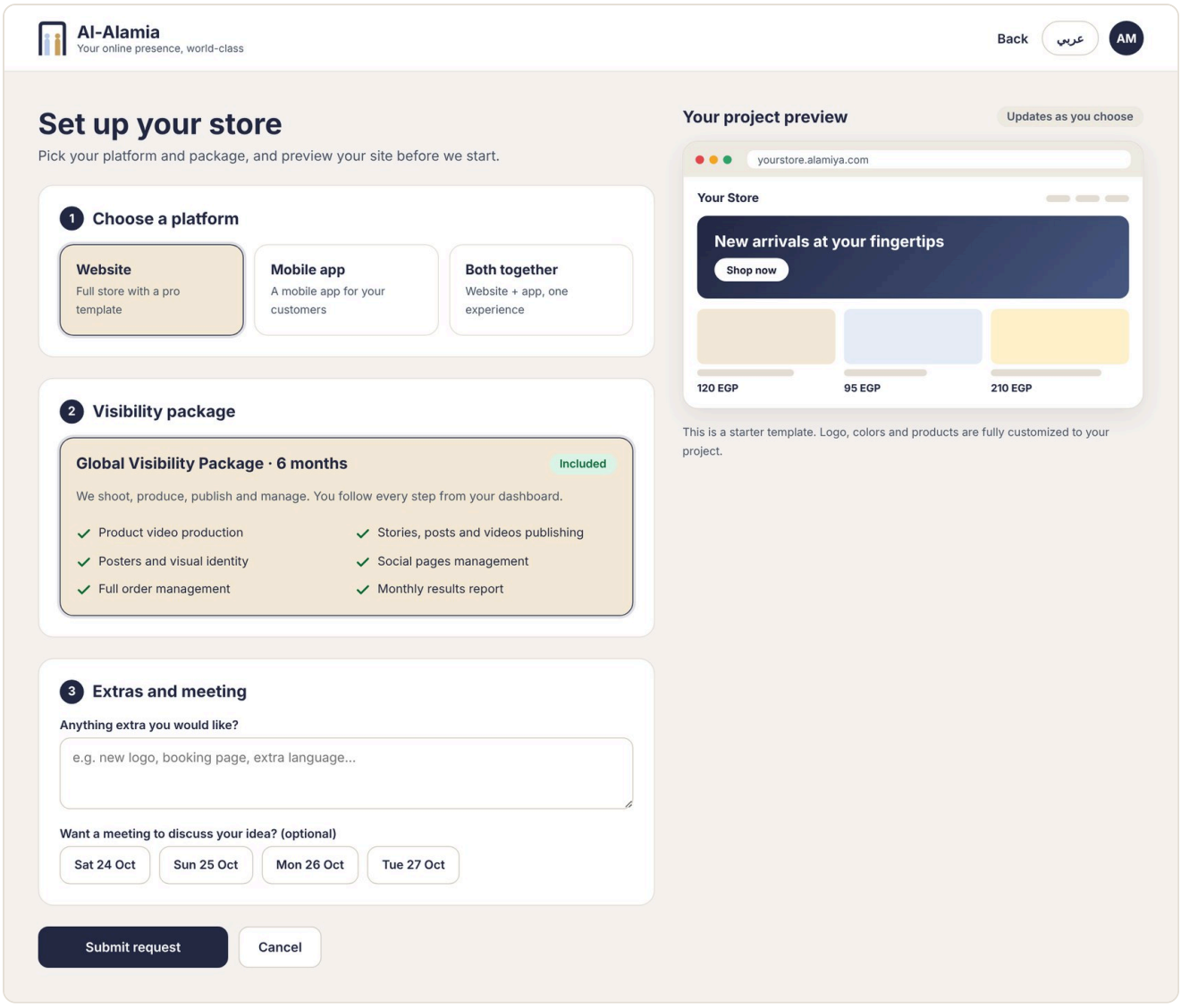
Sign in



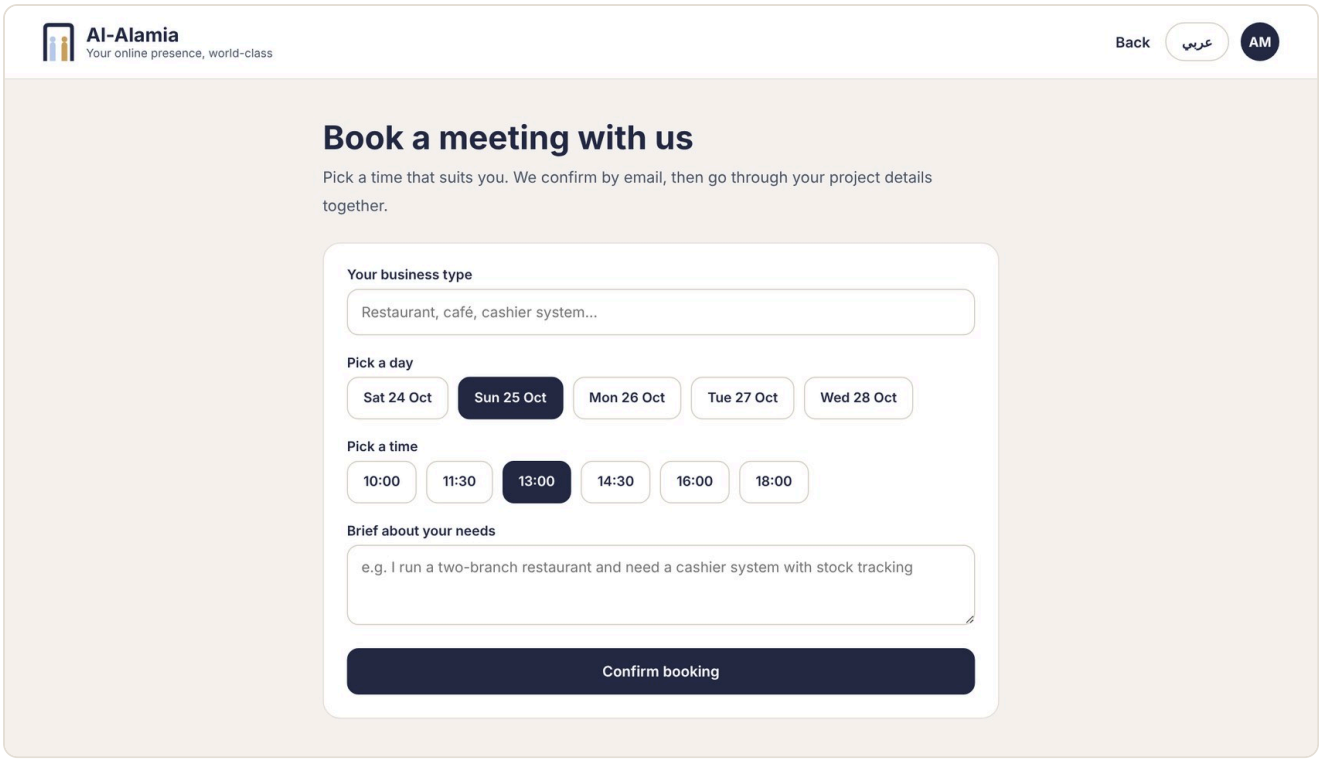
Choosing a path: client or freelancer



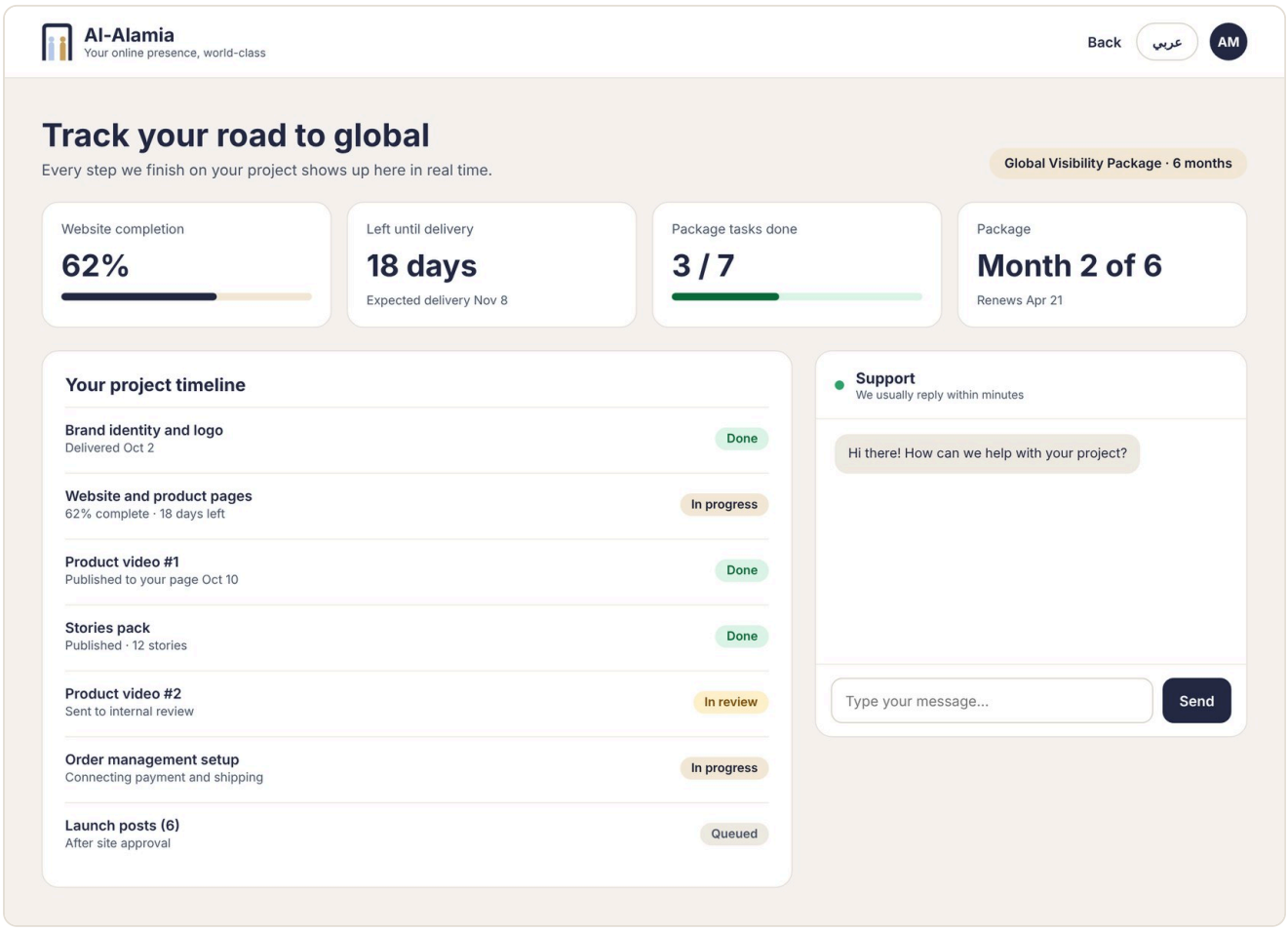
Choosing the project type



Store setup: platform, package and add-ons

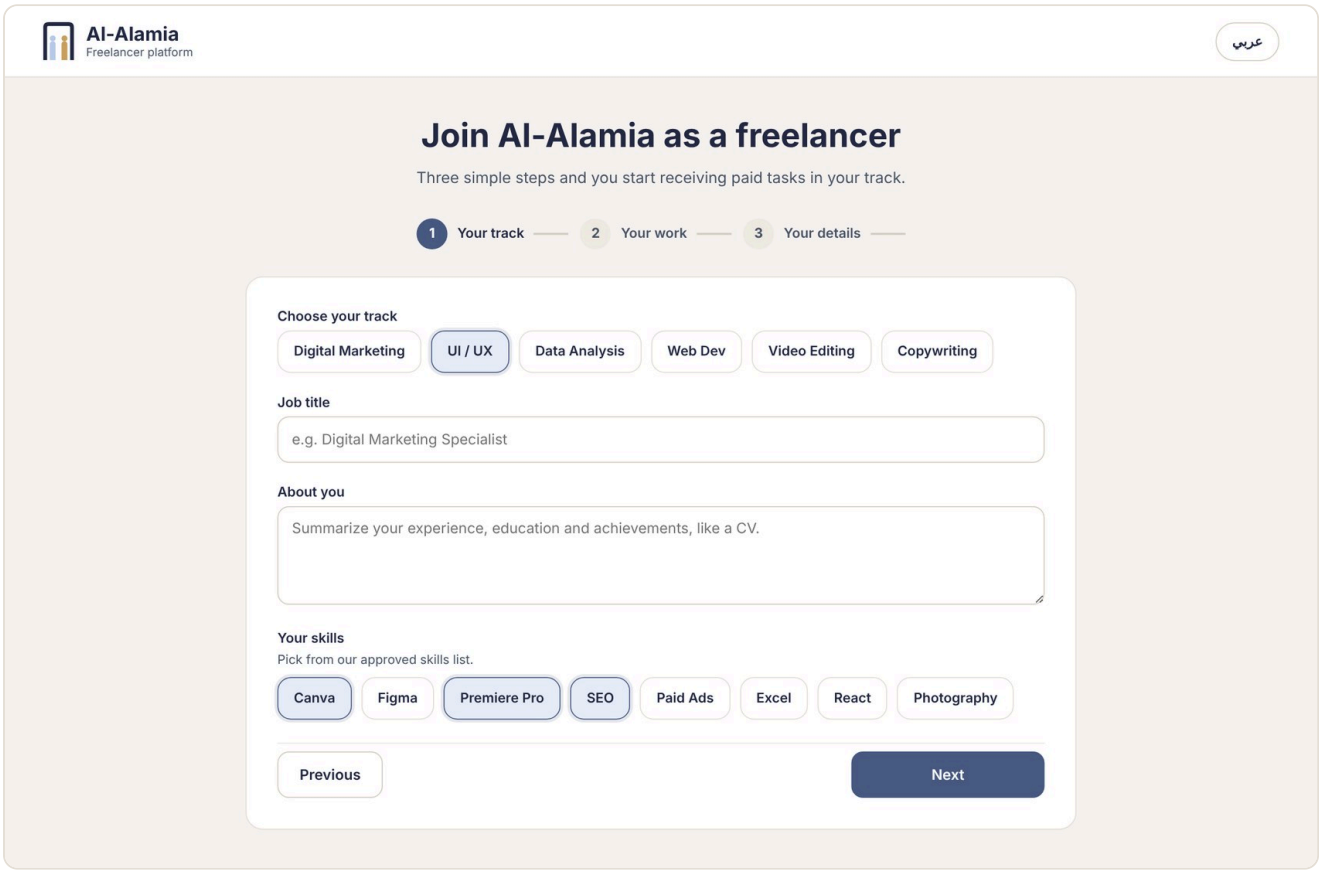


Booking a meeting (for customizations)

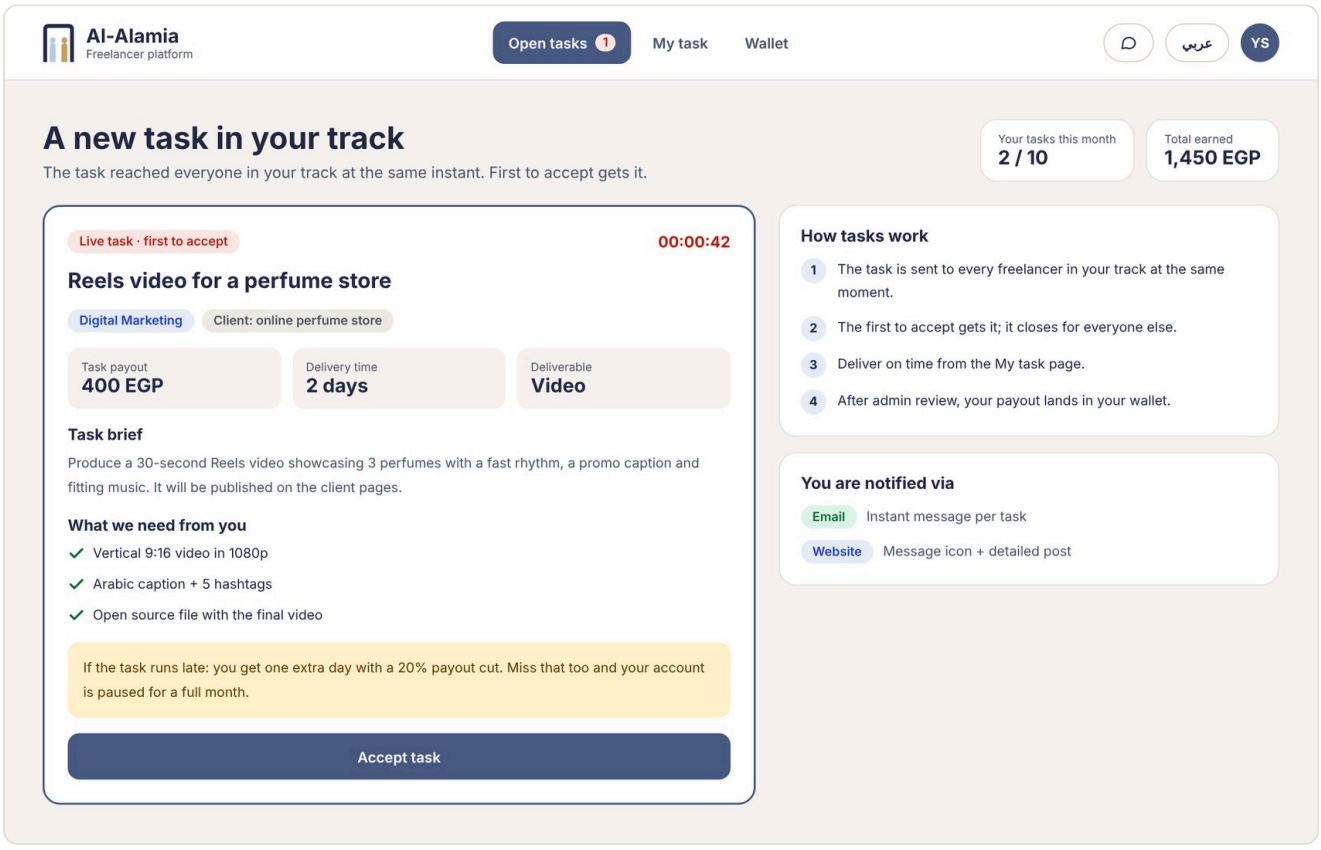


Project tracking and the support channel

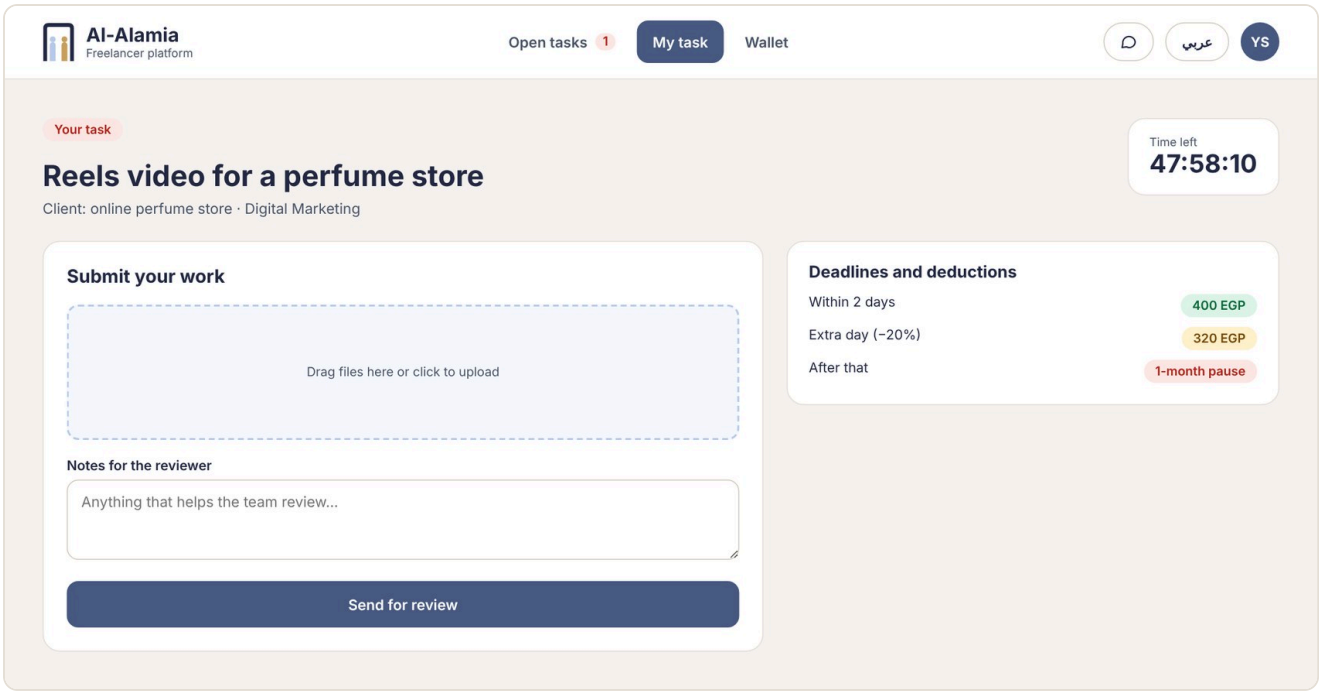
B) Freelancer journey



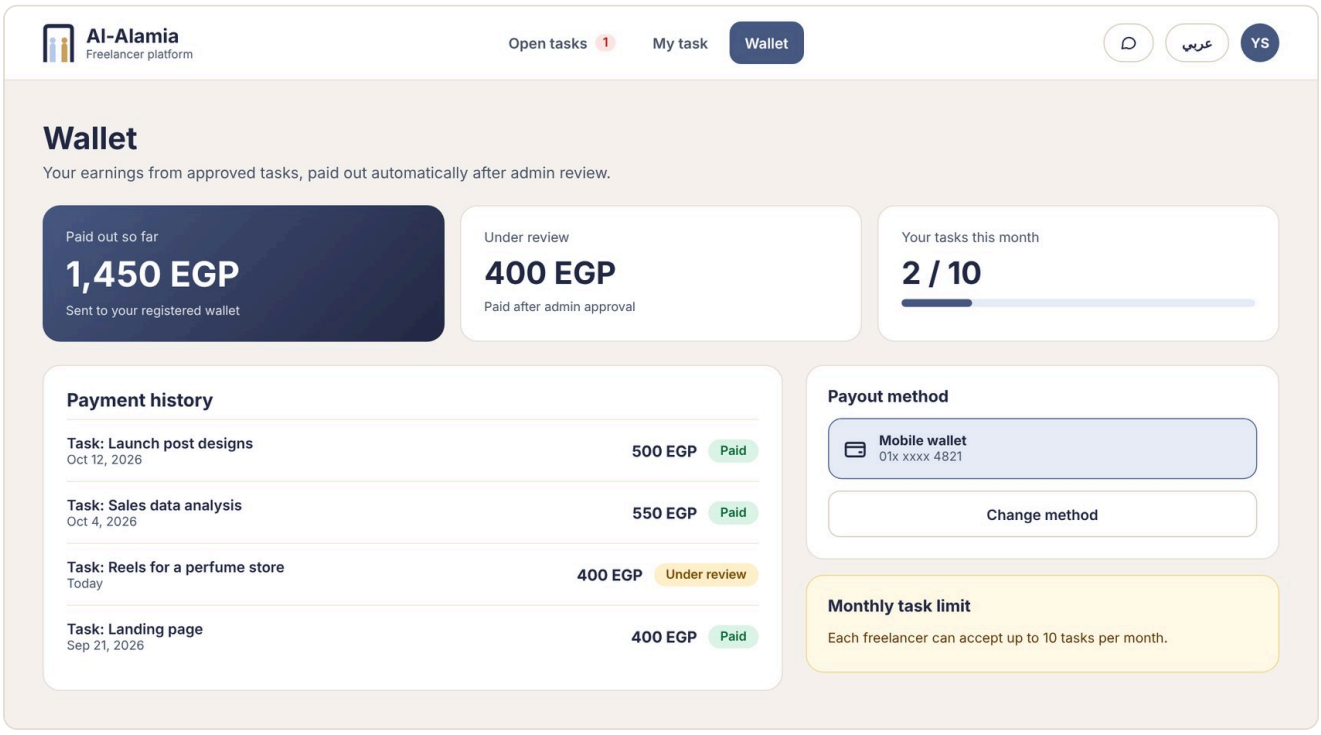
Three-step sign-up: track and skills



Task offers (first to accept wins)

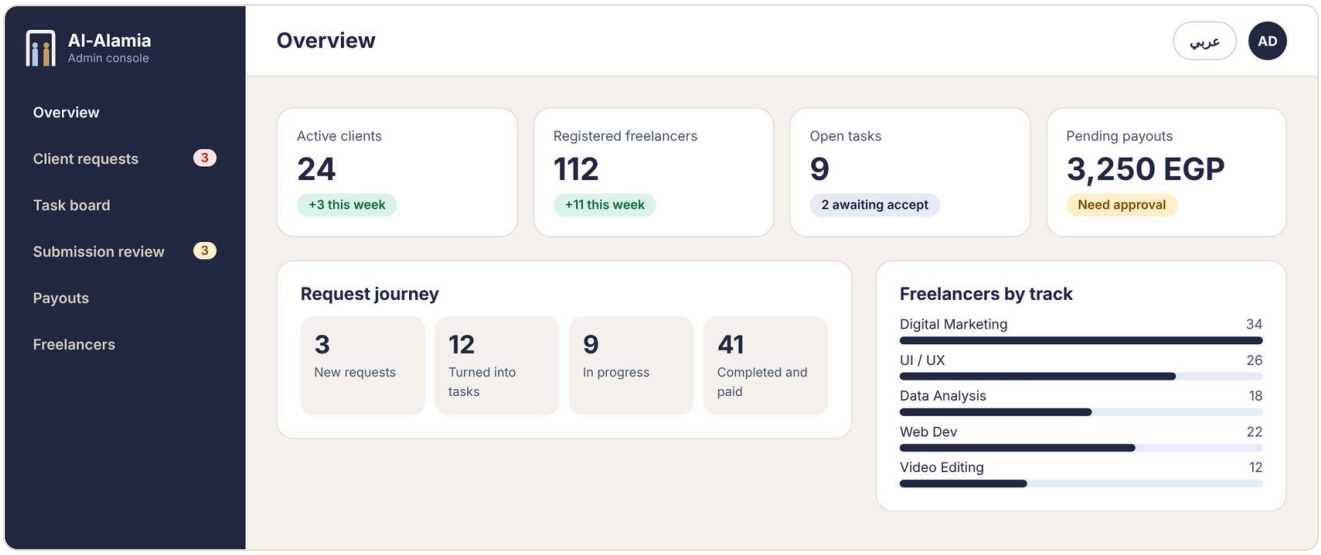


The current task and delivery

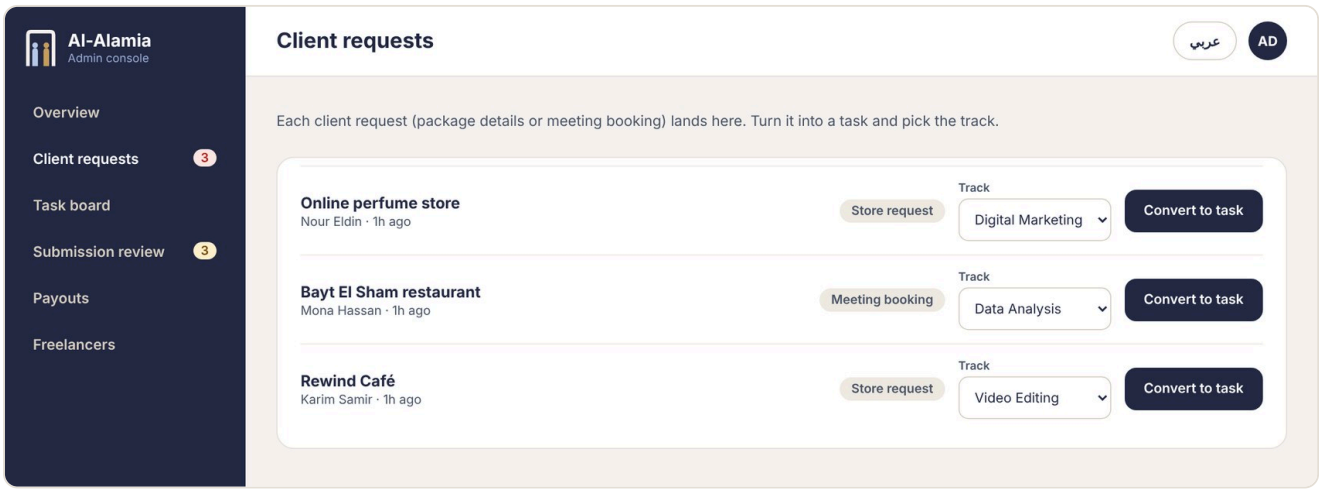


Wallet: the financial ledger

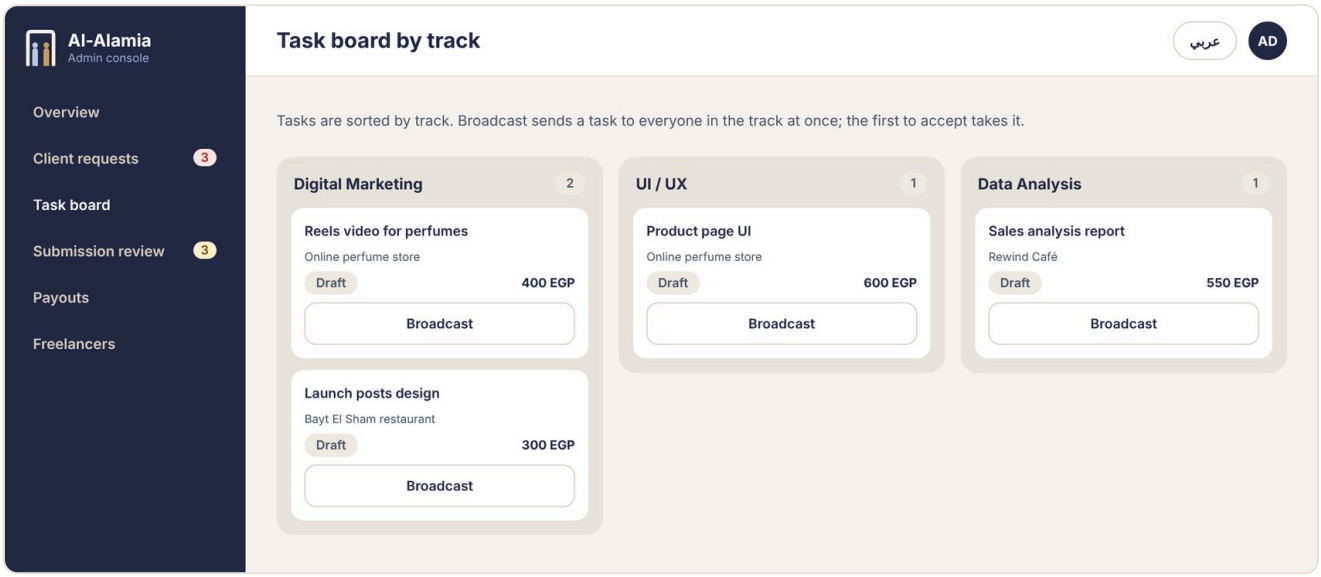
C) Admin console



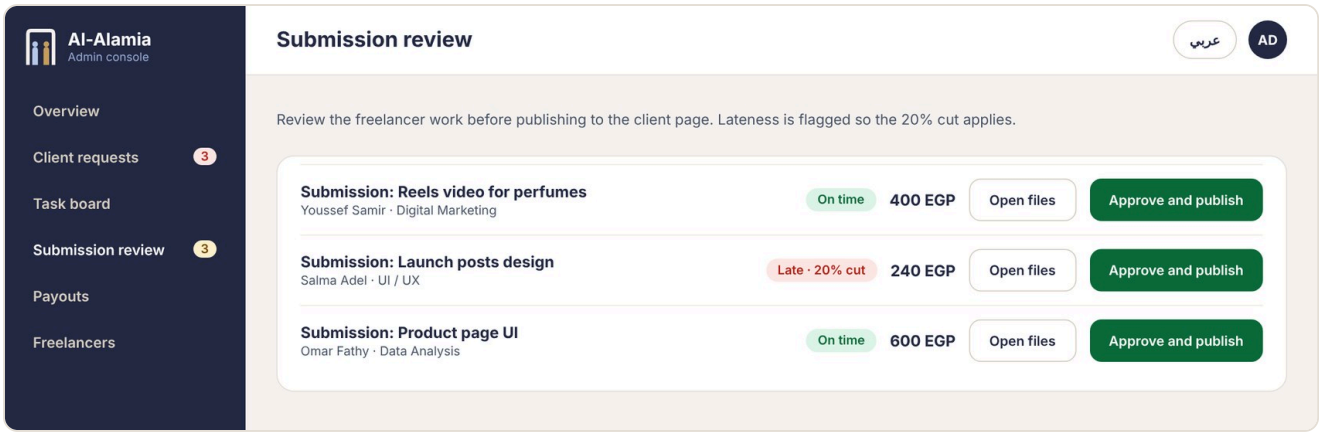
Overview



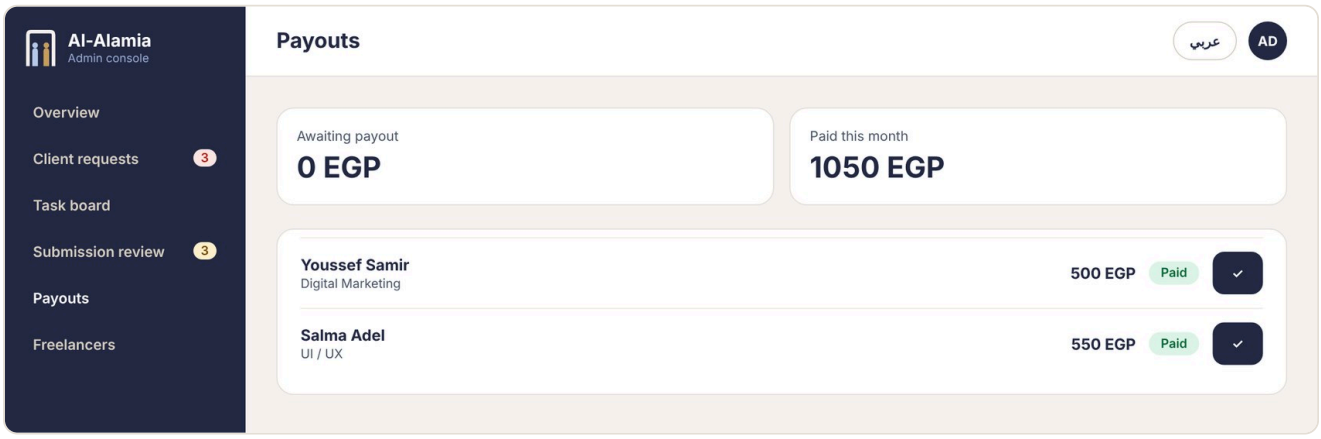
Client requests and turning them into tasks



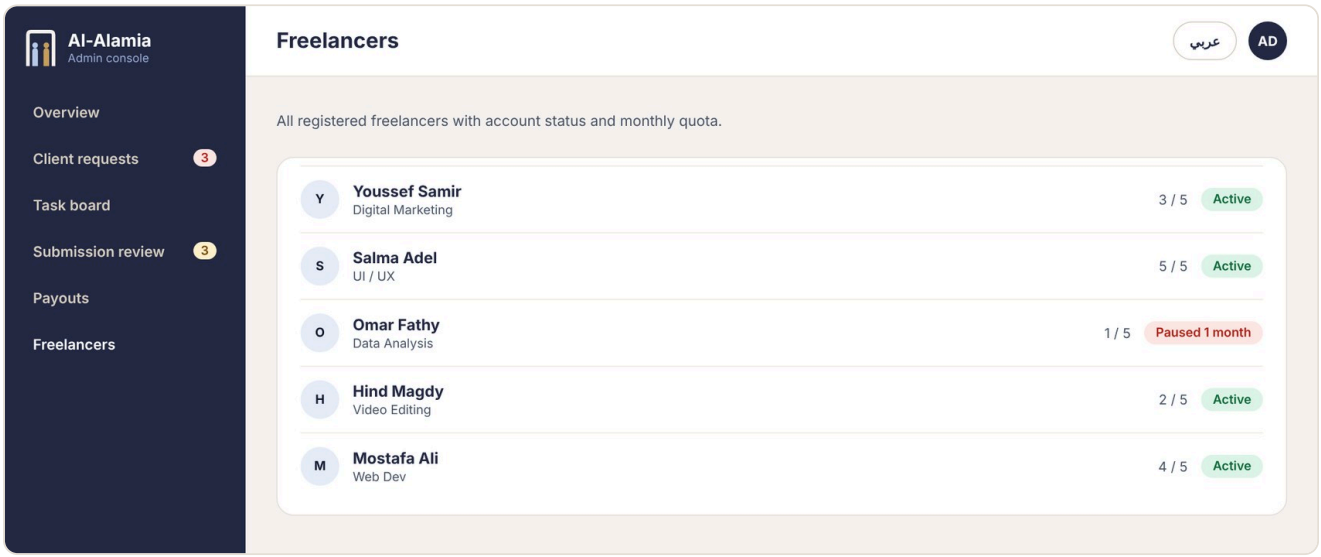
Task board by track



Submission review



The financial ledger



Freelancer management

11 Performance, security and quality

These are measurement targets we commit to and will check with tools; they are not figures already achieved. Actual results go into `docs/perf-report.md` at the end of week 5.

11.1 Performance targets

< 200ms
p95 for common read queries at 100 concurrent users

< 300ms
p95 for accepting a task (the most contended path)

< 250KB
First front-end bundle after gzip

≥ 90
Lighthouse score for performance and accessibility

Area	Approach
Database	Query-driven indexes (8.3), projection to DTOs instead of loading entities, `AsNoTracking` for reads, no N+1 (deliberate `Include` or split queries), keyset pagination
API	Response compression, `OutputCache` for catalog data (tracks, skills, services) invalidated on edit, fully async code with no blocking calls
Critical paths	Accept as one conditional update in a short transaction, an index-only monthly count, no network calls (e-mail) inside a transaction
Front end	Route-level code splitting, lazy image loading, TanStack Query caching, optimistic update on accept with rollback on 409
Measurement	k6 load scenarios (browsing offers, parallel accept), slow-query analysis with Serilog and execution plans

11.2 Security

- Passwords through Identity (PBKDF2 hashing and a strength policy), lockout after failed attempts, e-mail confirmation.
- Tokens: only `TokenHash` in the database, rotation on use, and the whole token chain revoked when an old token is replayed.
- Ownership checked in every use case; identifiers from the client are never trusted.
- Files: type allow-list, size limit, random storage names, served through an authorized endpoint, `X-Content-Type-Options` header.
- CORS limited to the front-end origin, security headers, HTTPS enforced outside development, secrets in environment variables and not in the repository.
- An OWASP Top 10 review as a checklist at the end of week 5, starting with Broken Access Control.

11.3 Test strategy

Level	What it covers	Tools
Unit (Domain)	State transitions, deduction and earning calculation, calendar-month calculation; every BR-xx rule has at least one test	`xUnit · FluentAssertions`
Integration (API + SQL)	The full journey, authorization, unique constraints, **the concurrent accept test (50 requests)**	`WebApplicationFactory · Testcontainers · Respawn`
Front end	Form components and the four states, the accept flow with a 409 error	`Vitest · Testing Library`
End to end	One golden scenario: client request → task → accept → delivery → accept → ledger	`Playwright`

Coverage target: at least 80% for the Domain and Application layers, and 100% on the critical rule paths (BR-04 to BR-12). CI fails if coverage drops or any test fails.

11.4 Code quality and operations

- **Standards.** A shared `.editorconfig`, Roslyn analyzers as errors, nullable enabled; ESLint + Prettier + `tsc --strict` on the front end.
- **Review.** Every merge goes through a pull request with at least one teammate's approval, and branches are short-lived.
- **CI.** GitHub Actions: build, tests, analysis, migrations applied to a clean database, front-end build.
- **Running it.** `docker compose up` starts SQL Server, the API and the front end together. A `/health` endpoint and structured logs with a correlation id.
- **Demo data.** A seeder creates sample accounts for the three roles, plus projects and tasks, for the final presentation.

12 Delivery plan and team ownership

The project starts on Wednesday 28 October 2026 and is handed in by **Saturday 5 December 2026**: five and a half weeks, with the last days kept as a buffer. Each member owns **a complete vertical module** (database, API, UI and tests). That cuts hand-offs between people and keeps responsibility clear.

12.1 Module ownership

Member	Owned module	Key deliverables
Mohamed Tawfik	Tasks: `Task`, `TaskOffer`, `TaskRequiredSkill`, `TaskStatusHistory`, `TaskReassignment`, `TaskFile`	Assignment and concurrent-accept engine, task state machine, task board
Menna Reda	Identity and freelancer: Identity, `RefreshToken`, profiles, tracks and skills, portfolio, `FreelancerBlock`	Authentication and roles, freelancer approval, portfolio, block check
Abdelrahman Khaled	Projects: `Service`, `Project`, `ProjectService`, `ProjectStatusHistory`, `Meeting`, marketing packages	Client journey, pricing, meetings, snapshot packages
Merna Abdelfattah	Delivery and finance: `TaskSubmission`, `TaskReview`, `TaskRevision`, `SubmissionFile`, `FreelancerLedgerEntry`	Review and revisions, late deduction and blocks, the ledger
Nahed Magdy	Files, support and notifications: `File`, `ProjectFile`, `Support*`, `Notification`, `EmailNotificationLog`	Secure file service, tickets, notifications and e-mail (outbox)

> **The split is a proposal** and can be swapped by team agreement. The fixed principle: each module has one owner responsible for finishing it, and everyone reviews each other's code.

Shared responsibilities. The design system and component library (built together in week 1), the solution skeleton and CI (Mohamed and Menna), demo data and documentation (everyone, each for their own module).

12.2 Timeline

Week	Goals	Milestone
1 28 Oct – 3 Nov **Foundation**	• Repository, CI, solution structure, code conventions • Migrations for all modules (one merged schema) • Identity, JWT, refresh tokens and roles • Design system, app shell, i18n and RTL • Freeze the OpenAPI contract per module • Seeder and Docker setup	**M1:** login works for all three roles, and the database builds from migrations on a clean machine
2 4 – 10 Nov **Flows**	• Freelancer sign-up and approval, portfolio • Service catalog, project creation, meetings • Admin pricing, task creation and publishing • Offer feed for freelancers • Secure file upload	**M2:** a client submits a project, the admin prices it and creates tasks, and they appear as offers to eligible freelancers
3 11 – 17 Nov **Engine**	• Concurrent accept and the monthly cap, plus the 50-request test • Delivery, review and revisions • Notifications and e-mail outbox • Support tickets	**M3:** the first-accept engine passes the 50-request test, and delivery and review work end to end
4 18 – 24 Nov **Rules and money**	• Deadline monitor: deduction, block, reassignment • Financial ledger • Marketing packages • Withdrawal and final-rejection paths • Front-end screens for every module	**M4:** the whole journey works from request to ledger, including late, rejected and withdrawn cases
5 25 Nov – 1 Dec **Polish**	• Performance tuning, k6 runs, query optimization • Integration tests and the golden E2E scenario • Accessibility, RTL polish, error and empty states • OWASP security review • Documentation, demo data, demo video	**M5:** a release candidate with a performance report and green tests
Buffer 2 – 5 Dec	Code freeze, critical fixes only, final presentation rehearsal, packaging the hand-in	**Hand-in** Saturday 5 Dec

Short, regular check-ins. A 15-minute daily stand-up (what I finished, what I will do, what blocks me) and a weekly review with a demo of whatever is complete. Each week is measured against its milestone, not against lines of code.

13 Risks and definition of done

13.1 Risk register

Risk	Impact	Response
Migration conflicts between five people changing the schema	High	The full schema is built in week 1 and merged once. After that, any change goes through a dedicated pull request with mandatory review.
Front end falling behind the API	High	The OpenAPI contract is frozen early, mock data is generated from it, and each member builds the UI for their own module.
Scope creep (requests for new features)	High	Every new idea goes to future work and enters only if something of the same size comes out.
Concurrency bugs in accept and the monthly cap	High	Conditional update plus the 50-request test in CI, and a second reviewer on this code.
An ambiguous rule (such as the monthly cap, 5 or 10)	Medium	Settled in the week-1 meeting, recorded in section 4, and implemented as a setting.
A short calendar: five and a half weeks, with the last days as buffer	Medium	Milestones M1 to M5 are checked weekly; if one slips, the next week's polish items are cut first, never the critical flows or their tests.

13.2 Definition of done

A task is closed only when all of the following hold:

- The code is merged after review and CI is green (build, tests, analysis).
- A unit test covers each business rule the task touches, and an integration test covers the new path.
- Permissions and ownership are checked, and errors carry stable, translated codes.
- The screen works in all four states, in Arabic and English, RTL and LTR.
- No query lacks a suitable index or has an N+1 problem, and the OpenAPI contract and affected docs are up to date.

13.3 Final acceptance criteria

#	Criterion
1	The golden scenario works completely, from the client's request to the earning appearing in the ledger.
2	The concurrent accept test passes: one winner out of 50 requests.
3	Late, rejected and withdrawn cases produce the deduction, block and reassignment described in section 7.
4	The performance targets (11.1) are measured and recorded, and anything missed is stated openly with a plan.
5	The UI works in Arabic and English without direction bugs and works on mobile, and the project starts with one command (`docker compose up`) with demo data ready.

14 Future work

These ideas are deliberately kept outside the delivery window. The current design leaves each of them possible without a rebuild, which is why every item names the foundation that already exists for it.

Idea	Value	Foundation already in the design
Real payments	Pay out earnings and collect client money	An append-only ledger; a payment gateway can sit behind an `IPaymentGateway` interface
Skill assessment tests	Better approval quality, less manual review	Approval flow, tracks and skills are in place
Real-time notifications and chat	A faster experience than polling	The notification table and event points; SignalR is added as a transport
Ratings and ranking	Order offers by quality, not only by speed	Task and review history as the data source
Smart task matching	Suggest the best fit instead of broadcasting to everyone	Task skills, freelancer skills and historical acceptance rate
Analytics and reports	Admin dashboards and export	History tables with precise timestamps allow duration calculations
Mobile app	Faster access for freelancers	A standalone API and a responsive React UI
Splitting into services	Scale with load	Modules with clear boundaries and public interfaces between them

A Appendix: glossary and enum values

Glossary

Term	Meaning
Track	A freelancer's primary specialty (such as UI/UX or Backend). Tasks are routed by it.
Skill	A specific ability that can belong to more than one track and be required by a task.
Task Offer	An offer sent to an eligible freelancer; it becomes their task only if they are the first valid accept.
Ledger	A financial record of entries (earning, deduction, adjustment) with no money transfer.
Snapshot	A copy of a package's details and price at purchase time, unaffected by later edits to the original.
Outbox	Writing the message to the database inside the event's transaction and sending it later, so it is neither lost nor able to block the operation.
Keyset pagination	Paging by the last key value instead of an offset; performance stays flat as tables grow.
Rawad (Digital Egypt Pioneers)	The initiative the team belongs to.

Enum values

Enum	Values
ApprovalStatus	`1 Pending · 2 Approved · 3 Rejected`
ProjectStatus	`1 Submitted · 2 UnderReview · 3 Priced · 4 InProgress · 5 Completed · 6 Cancelled`
MeetingStatus	`1 Pending · 2 Approved · 3 Rejected`
TaskStatus	`1 Draft · 2 Open · 3 Assigned · 4 Submitted · 5 RevisionRequested · 6 Completed · 7 Cancelled`
OfferStatus	`1 Pending · 2 Accepted · 3 Declined · 4 Closed`
SubmissionStatus	`1 Submitted · 2 Accepted · 3 RevisionRequested · 4 Rejected`
ReviewDecision	`1 Accept · 2 RequestRevision · 3 RejectFinal`
RevisionStatus	`1 Open · 2 Completed`
LedgerEntryType	`1 Earning · 2 LateDeduction · 3 ManualAdjustment`
PackageStatus	`1 Active · 2 Expired · 3 Cancelled`
TicketStatus / Priority	`1 Open · 2 InProgress · 3 Closed / 1 Low · 2 Normal · 3 High`

EmailStatus	1 Pending · 2 Sent · 3 Failed